A Project Report on

ALGOPILOT

(Algorithm Visualizer and Execution Platform)

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

By

Shikshita Yadav: 22106092

Disha Waghmare: 22106033

Pratiksha Pathak: 22106127

Under the Guidance of

Dr. Jaya Gupta

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "Algopilot(Algorithm Visualizer and Execution Platform)" is a bonafide work of **Shikshita Yadav(22106092), Disha Waghmare(22106033), Pratiksha Pathak(22106127)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning**).

Dr. Jaya Gupta	Prof. Sayali Badhan
Guide	Project Co-ordinator

Dr. Jaya Gupta	Dr. Uttam Kolekar
Head of Department	Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for B.E.

This project report entitled ***Algopilot(Algorithm Visualizer and Execution Platform)*** by ***Shikshita Yadav, Disha Waghmare, Pratiksha Pathak*** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Science & Engineering (Artificial Intelligence & Machine Learning)***, ***2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Shikshita Yadav (22106092)

Disha Waghmare (22106033)

Pratiksha Pathak (22106127)

Date:

Abstract

This project presents AlgoPilot, an Android-based mobile application developed to enhance the learning of Data Structures and Algorithms (DSA) through interactive visualization. Traditional learning methods such as textbooks and static diagrams often make it difficult for students to understand the step-by-step execution of algorithms. AlgoPilot addresses this challenge by using custom-built animations synchronized with pseudocode, allowing learners to clearly observe how algorithms perform operations such as comparisons, swaps, insertions, and deletions. This approach helps students connect theoretical concepts with practical execution. The application also provides interactive controls that allow users to adjust the execution speed, pause the algorithm, and move forward or backward through the steps. These features support self-paced learning and make complex algorithms easier to analyze. In addition, AlgoPilot works offline after installation, ensuring accessibility for students without continuous internet connectivity. Its modular design also allows additional algorithms and features to be integrated in the future. AlgoPilot supports the United Nations Sustainable Development Goals (SDGs) by promoting technology-enabled education. It contributes to SDG 4 (Quality Education), SDG 9 (Industry, Innovation, and Infrastructure), and SDG 10 (Reduced Inequalities) by providing accessible and interactive algorithm learning through mobile technology.

Keywords: Algorithm Visualization, Data Structures and Algorithms (DSA), Mobile Learning, Interactive Education, Pseudocode Synchronization, Offline Accessibility, Modular Design, Self-paced Learning.

Contents

1. Introduction 1
1.1 Need for the Application 2
1.2 What Is DSA and Algorithm Visualization? 2
1.3 Overview of Android Studio 3
2. Survey/ Existing System 4
3. Limitation of Existing System 10
4. Problem Statement and Objective 12
4.1 Problem Statement 12
4.2 Objective 13
4.3 Scope 13
5. Proposed System 15
6. Experimental Setup 21
7. Results and Discussion 24
8. Conclusion and Future Work 43
8.1 Conclusion 43
8.2 Future Scope 44
References 45
Appendix 47
Publication 50

LIST OF FIGURES

Figure 5.1: Proposed System 15
Figure 5.1.1: Data Flow Diagram 16
Figure 5.2.1: Use Case Diagram 17
Figure 5.3.1: Sequence diagram 18
Figure 5.4.1: Activity diagram 19
Figure 7.1: Algorithms Implemented 24
Figure 7.2: Sorting Homepage 25
Figure 7.3: Bubble Sort 25
Figure 7.4: Selection Sort 26
Figure 7.5: Insertion Sort 27
Figure 7.6: Quick Sort 28
Figure 7.7: Merge Sort 29
Figure 7.8: Tree Homepage 30
Figure 7.9: AVL Tree 31
Figure 7.10: Insertion in AVL tree 31
Figure 7.11: BST 32
Figure 7.12: Searching in BST 32
Figure 7.13: Graph Homepage 33
Figure 7.14: BFS 34
Figure 7.15: DFS 35
Figure 7.16: Dijkstras 36
Figure 7.17: Bellman Ford 37
Figure 7.18: Kruskal's algorithm 38
Figure 7.19: Prim's algorithm 39

LIST OF TABLES

Table 2.1: Research Gap 7
Table 7.1: Performance Analysis 40

LIST OF GRAPHS

Graph 7.1: Sorting Algorithms Vs Time Complexity 41
Graph 7.2: Tree Algorithms Vs Time Complexity 41
Graph 7.3: Graph Algorithms Vs Time Complexity 42

ABBREVAION

DSA	*Data Structures and Algorihtms*
IDE	*Integrated Developmet Environment*
JSAV	*JavaScript Algorihtm Visulaization*
JHAVE	*Java Hosted Algorithm Visualization Environment*
JAWAA	*Java and Web-based Algorithm Animation*
BFS	*Breadth First Search*
DFS	*Depth First Search*

Chapter 1

Introduction

1. Introduction

Data Structures and Algorithms (DSA) form the foundation of computer science and play a crucial role in efficient programming, problem solving, and software development. Data structures define how data is organized, stored, and accessed, while algorithms provide the step-by-step procedures used to process that data. A strong understanding of DSA is essential for building optimized applications, improving computational efficiency, and performing well in technical interviews and competitive programming. However, the abstract nature of these concepts often makes them difficult for students to understand. Traditional learning methods such as textbooks, lectures, and static diagrams mainly focus on theoretical explanations and fail to clearly demonstrate how algorithms behave during actual execution. Since algorithms involve dynamic operations like comparing values, swapping elements, traversing nodes, or updating structures, students often struggle to visualize these processes mentally, leading to gaps in understanding and reduced problem-solving confidence.

To address this issue, this project proposes the development of a mobile application using Android Studio, the official Integrated Development Environment (IDE) for Android development. The application aims to simplify algorithm learning by using animated

Visualizations that demonstrate how algorithms execute step by step in real time. By allowing learners to observe sorting techniques, tree traversals, and graph algorithms visually, the application helps them understand how data changes at each stage of execution. This interactive approach bridges the gap between theoretical knowledge and practical understanding by converting abstract concepts into clear visual representations. Additionally, the application encourages active and self-paced learning, enabling students to explore algorithms at their own speed using controls such as play, pause, and step-by-step execution. This makes the learning process more engaging, intuitive, and effective, especially for beginners.visualizations that demonstrate how algorithms execute step by step in real time. By allowing learners to observe sorting techniques, tree traversals, and graph algortihms visually, the application helps them understand how data changes at each step of execution. This interactive approach bridges the gap between theoretical knowledge and practical understanding. Additionally, the application encourages active learning by enabling students to explore algorithms at their own pace, making the learning process more engaging, intuitive, and effective.

1.1 Need for the Application

Most existing algorithm visualization tools are web-based and require desktop systems along with continuous internet connectivity. This limits accessibility for students who rely primarily on mobile devices or who may not always have a stable internet connection. In many cases, learners cannot access these tools while traveling or in low-resource environments. Therefore, there is a strong need for a portable, offline, and interactive learning tool that allows students to study algorithms anytime and anywhere. Such a solution is especially beneficial for students preparing for academic examinations, competitive programming contests, and technical interviews, where consistent practice and clear understanding are essential. The proposed application fulfills this need by providing a mobile-based platform that ensures flexibility, accessibility, and ease of use.

1.2 What Is DSA and Algorithm Visualization?

Data Structures and Algorithms (DSA) are core concepts that enable efficient data handling and problem solving in computer science. Data structures such as **a**rrays, linked lists, stacks, queues, trees, and graphs provide organized ways to store and manage data, while algorithms define the logical steps required to manipulate that data to achieve a desired outcome. Understanding how these components work together is essential for writing efficient programs.

Algorithm visualization refers to the graphical representation of algorithm execution, where the internal working of algorithms is shown through animations and visual elements. Instead of simply reading code, learners can observe how algorithms perform operations such as comparisons, swaps, insertions, deletions, and traversals. This approach helps students clearly understand the flow of logic, making it easier to grasp complex concepts. Visualization also improves conceptual clarity, retention, and problem-solving ability, as students can see how each step contributes to the final result.

1.3 Overview of Android Studio

Android Studio is the official development environment used for building Android applications. It provides a comprehensive set of tools including an advanced code editor, layout designer, debugging tools, and device simulators, which assist developers in designing, developing, and testing applications efficiently. Android Studio supports modern development practices and ensures compatibility across different Android devices and screen sizes.

In this project, Android Studio was used to develop a Java-based mobile application with a modular structure. This modular design allows different algorithms to be implemented as separate components, making the application easier to maintain and expand. The application is designed to be lightweight, efficient, and user-friendly, ensuring smooth performance on various devices. By leveraging Android Studio, the project delivers a portable and accessible learning tool that enables students to understand algorithms through interactive visualizations anytime and anywhere.

Chapter 2

Literature Survey/ Existing System

2. Survey/ Existing System

[1] Algorithm Visualization on Data Structures Along with AI Chatbot (2025)

This paper proposes a system that integrates algorithm visualization with an AI chatbot to improve the learning of data structures and algorithms. The system visually demonstrates how algorithms operate on data structures, allowing students to observe steps such as comparisons, insertions, deletions, and traversals in real time. In addition to visualization, the platform includes an AI chatbot that assists learners by providing explanations and answering questions related to algorithm logic. This interactive approach helps students better understand complex algorithm concepts while also offering instant support when difficulties arise. The study concludes that combining visualization with AI-based assistance improves student engagement and enhances the effectiveness of algorithm learning.

[2] Demystifying Complex Algorithms: A Visual Guide Through the DSA Visualizer (2024)

This research by Jeet Matalia, Shreya Kathiriand, and Priyanka Patel (2024) presents a DSA Visualizer designed to simplify complex algorithms through graphical representations. The system provides visual demonstrations of algorithm execution to help learners understand the internal working of data structures and algorithms. It

allows users to observe operations such as comparisons, swaps, and traversals in real time. The study highlights that visualization helps students grasp complex algorithmic concepts more easily. It also improves logical thinking and problem-solving skills.

[3] Application of Algorithm Visualization Techniques in Teaching Computer Data Structure Course (2024)

In this study, it explores the use of algorithm visualization techniques in teaching data structure courses. The research highlights that visual demonstrations help students understand how algorithms manipulate data structures during execution. By observing the flow of operations, learners can better grasp complex programming concepts. The study concludes that visualization techniques improve student engagement and support more effective teaching of computer science subjects.

[4] The Efficacy of Animation and Visualization in Teaching Data Structures: A Case Study (2024)

This paper examines the effectiveness of animation and visualization techniques in teaching data structures. The research compares traditional teaching methods with visualization-based learning approaches. The authors found that students who learned using animated visualizations showed better understanding of data structure operations and algorithm flow. Visualization helped learners observe how data changes during execution and improved their ability to analyze algorithm behavior. The study concludes that animation-based learning tools significantly enhance conceptual clarity and student engagement in computer science education.

[5] Visualizing Algorithms in the Field of Education: A Comprehensive Review (2023)

The study by V. Gupta and D. Rao (2023) provides a comprehensive review of different algorithm visualization techniques used in education. The authors analyzed various visualization tools and platforms that support interactive learning of algorithms. The research concludes that visual learning methods significantly improve student engagement and understanding compared to traditional teaching methods. It also emphasizes the importance of integrating visualization tools into computer science education. These tools help students better understand algorithm flow and data manipulation processes.

[6] Research on Graphic Interactive Visual Online Teaching of Intelligent Algorithm Course (2023)

The study by G. Lou, W. Sufang, and H. Shen (2023) investigates the use of graphical interactive visualization in teaching intelligent algorithm courses. The researchers developed an online visual teaching system that allows students to interact with algorithm demonstrations. This approach encourages active learning and improves student participation. The study

concludes that interactive visualization tools enhance understanding of algorithm concepts and make learning more engaging.

[7] Evaluating Software Algorithm Visualization Projects (2023)

In this study, A. Kumari, N. Bohra, and P. Sangwan evaluated various algorithm visualization software projects used in computer science education. The research analyzed different visualization tools to determine their effectiveness in helping students understand algorithm behavior. The authors found that visualization-based learning tools significantly improve students' conceptual understanding and engagement. The study also emphasizes the importance of designing user-friendly visualization systems that support interactive learning.

[8] Research and Application of Animation Visualization Based on HTML5 Algorithm (2023)

This research focuses on developing animation-based algorithm visualization using HTML5 technology. The study demonstrates how modern web technologies can be used to create interactive visual learning environments. Animated visualizations help learners observe algorithm steps clearly and understand the logical flow of operations. The research concludes that animation-based visualization improves the effectiveness of programming education.

[9] Algorithm Visualisation—Modern Web-Based Visualization of Sorting and Searching Algorithms (2022)

This study by Ashish Kumari et al. (2022) focuses on a modern web-based visualization platform designed to demonstrate sorting and searching algorithms. The system visually represents algorithm operations such as comparisons, swaps, and iterations, helping students understand how algorithms process data step by step. The research highlights that visualization tools can significantly improve the comprehension of algorithmic concepts compared to traditional static learning methods.

[10] Algovis – An Enhanced Way to Visualize Algorithms (2022)

This research introduces *AlgoVis*, an enhanced algorithm visualization tool designed to improve the learning experience for students. The system provides animated demonstrations of algorithms along with interactive features that allow users to follow each step of execution. The study emphasizes that visualization-based tools make algorithm learning more engaging and effective. It also shows that such tools help students understand complex algorithm logic more easily.

[11] Developing Android Mobile Application Using Java and Eclipse: An Application (2017)

The research conducted by Ş. Kocakoyun (2017) discusses the process of developing Android mobile applications using the Java programming language and the Eclipse development

environment. The study explains the design, development, and implementation of a mobile application for educational purposes. It highlights how Android platforms can support interactive learning tools and provide flexible access for students. The research also demonstrates that mobile applications can enhance learning by allowing users to access educational resources anytime and anywhere.

[12] Creating Engaging Online Learning Material with the JSAV Algorithm Visualization Library (2015)

The study by Ville Karavirta and Clifford A. Shaffer (2015) introduces the JSAV (JavaScript Algorithm Visualization) library used to create interactive algorithm visualizations for educational purposes. The library enables developers and educators to design animations that demonstrate algorithm operations clearly. It supports interactive learning by allowing students to manipulate data and observe algorithm behavior. The research highlights that such visualization libraries improve student engagement and make algorithm learning more effective.

[13] Teaching Sorting and Searching Algorithms Through Simulation-Based Learning (2014)

This study by G. Tuparov, D. Tuparova, and V. Jordanov (2014) focuses on teaching sorting and searching algorithms using simulation-based learning tools. The research developed interactive modules that allow students to observe how algorithms perform operations such as comparisons and swaps. By simulating algorithm execution, learners can clearly understand each step of the process. The study found that simulation-based visualization improves student engagement and helps beginners grasp algorithm concepts more effectively. It also enhances logical thinking and problem-solving abilities.

[14] JHAVÉ: Supporting Algorithm Visualization (2005)

In this study, T. L. Naps (2005) presented the JHAVÉ (Java Hosted Algorithm Visualization Environment) framework for supporting algorithm visualization in computer science education. The system integrates various visualization tools that allow students to interact with algorithms and observe their execution step by step. JHAVÉ also includes features such as quizzes and interactive exercises to enhance learning. The research concludes that visualization environments significantly improve conceptual understanding of complex algorithms.

[15] Web-Based Animation of Data Structures Using JAWAA (1998)

The research by W. C. Pierson and S. H. Rodger (1998) introduced JAWAA (Java and Web-based Algorithm Animation), a system for visualizing data structures and algorithms through web-based animations. The tool allows instructors and students to create animated demonstrations of operations on arrays, stacks, queues, and other data structures. These

animations help learners understand how algorithms modify data during execution. The study highlights that visual representations make algorithm behavior easier to understand than static code explanations.

Table 2.1: Research Gap

Research Paper	Year	Algorithm Used	Gap
Algorithm Visualization on Data Structures Along with AI Chatbot	2025	Sorting, Searching	Web-based, no offline capability, and no modular expansion
Demystifying Complex Algorithms: A Visual Guide Through the DSA Visualize	2024	Advanced Algorihtms	Web-only; no mobile/offline features.
Application of Algorithm Visualization Techniques in Teaching Computer Data Structure Course	2024	Data Structures and Algorithms	Instruction geared more toward desktops; lacks mobile or offline access.
The Efficacy of Animation and Visualization in Teaching Data Structures	2024	Sorting, Recursion, Graphs	Limited coverage of algorithms; no offline support.
Visualizing Algorithms in the Field of Education: A Comprehensive Review	2023	Sorting, Tress, Graphs	Survey only; no practical implementation
Research on Graphic Interactive Visual Online Teaching of Intelligent Algorithm Course	2023	Intelligent Algorithms and DS	Requires online connectivity; not focused on mobile or offline use.
Evaluating Software Algorithm Visualization Projects	2023	General Algorithm Visualization	Focuses only on evaluation; lacks mobile-based and offline implementation.
Research and Application of Animation Visualization Based	2023	General Algorithm Animation	Uses only web technologies; it does not have a mobile or modular extension.

on HTML5 Algorithm			
Algorithm Visualisation Modern Web-Based Visualization of Sorting and Searching Algorithms	2022	General Algorithm Animation	Limited to internet access alone; not mobile or offline compatible.
Algovis-An Enhanced Way to Visualize Algorithms	2022	Sorting, Searching	Focuses on improvement without a modular approach in mobile design.
Developing Android Mobile Application Using Java and Eclipse: An Application	2017	Android activity lifecycle & event-driven programming	Lack of practical, step-by-step, and scholarly guidelines for beginners.
Creating Engaging Online Learning Material with the JSAV Algorithm Visualization Library	2015	General Algorithms	Web-based library; requires setup and lacks standalone mobile/offline use.
Teaching Sorting and Searching Algorithms Through Simulation-Based Learning	2014	Sorting, Searching	Simulation limited to desktop learning; no mobile or offline support.
JHAVÉ: Supporting Algorithm Visualization	2005	General Algorithm Visualization	Framework-based approach; lacks portability and real-time interactivity.
Web-Based Animation of Data Structures Using JAWAA	1998	Data Structures (Arrays, Stacks, Queues)	Web-based and outdated;lacks modern UI and mobile compatibility

Chapter 3

Limitation of Existing System

3. Limitation of Existing System

1. **Web or Desktop Dependency**
   - Many existing systems are web-based and depend on browsers and internet connectivity.
   - This restricts usage in areas with poor or no internet access and limits flexibility.
2. **Lack of Offline Access and Portability**
   - Most tools do not support offline functionality and are designed for desktop use.
   - Students cannot access them anytime and anywhere, reducing learning convenience.
3. **Limited Mobile Compatibility**
   - Existing tools are not optimized for mobile devices or smartphones.
   - This reduces accessibility for modern learners who prefer mobile-based learning.
4. **Limited Algorithm Coverage**
   - Many systems focus only on basic algorithms like sorting and searching.
   - They do not provide a complete platform covering trees, graphs, and advanced algorithms.

5. **Lack of Modular and Scalable Design**
    - Most tools are not modular, making it difficult to add new features or algorithms.
    - They are often developed as prototypes rather than scalable educational platforms.
6. **Low Interactivity and Control**
    - Visualization is often passive, with limited user interaction.
    - Features like pause, step-by-step execution, and speed control are missing or limited.
7. **Complex or Outdated User Interface**
    - Some systems use outdated technologies or complex UI designs.
    - This reduces usability and makes them less beginner-friendly.
8. **Focus on Theory over Practical Use**
    - Many research works focus on evaluation or theoretical frameworks.
    - They lack practical, user-friendly implementations for real-world learning.
9. **Requirement of Technical Setup**
    - Some tools require installation, coding knowledge, or configuration.
    - This makes them difficult for beginners to use effectively.
10. **Limited Integration of Advanced Features**
    - Few systems include AI-based assistance or intelligent support.
    - Even when present, they lack proper mobile and offline integration.
11. **Requirement of Technical Setup**
    - Some tools require installation, coding knowledge, or configuration.
    - This creates difficulty for beginners and reduces accessibility.
12. **Limited Integration of Advanced Features**
    - Few systems include AI-based assistance or intelligent support.
    - Even when present, they lack proper mobile and offline integration.

Chapter 4

Problem Statement and Objective

4. Problem Statement and Objective

4.1 Problem Statement

Learning Data Structures and Algorithms (DSA) is often challenging for students due to its abstract and theoretical nature, as traditional methods like textbooks and static pseudocode fail to clearly demonstrate the step-by-step execution and real-time behavior of algorithms. Although various visualization tools exist, most are web-based or desktop-dependent, requiring continuous internet access and lacking mobile compatibility, offline support, and modular design. These limitations reduce accessibility and effectiveness, especially for students who rely on smartphones or have limited resources. Therefore, there is a need for a mobile-based, interactive, and offline-capable solution like AlgoPilot, which provides real-time animations, synchronized pseudocode, and step-by-step visualization to help learners better understand algorithm flow in an engaging and intuitive manner.

4.2 Objective

- **To develop an interactive and mobile-friendly learning platform:**

The main objective is to create an Android-based application that allows students to visualize algorithms step by step, making learning more engaging and accessible anytime and anywhere.

- **To enhance conceptual understanding through visualization:**

The system converts complex algorithm logic into real-time visual simulations, helping students understand how each step of the algorithm works.

- **To support self-paced and independent learning:**

The application allows users to control execution using features like pause, resume, and step-by-step navigation, enabling detailed analysis of algorithms.

- **To ensure offline accessibility and portability:**

The application is designed to work without internet connectivity, making it useful for students in remote or low-resource areas.

- **To provide a scalable and modular design:**

The system supports easy addition of new algorithms and features, ensuring long-term usability and expansion.

- **To improve user experience with a simple interface:**

The application focuses on a clean, intuitive, and beginner-friendly design to enhance usability.

- **To bridge the gap between theory and practical implementation:**

The system helps students connect theoretical knowledge with practical understanding, improving their problem-solving skills.

4.3 Scope

The scope of this project is to develop a mobile-based application (AlgoPilot) that enhances the learning of Data Structures and Algorithms (DSA) through interactive visualization. The application focuses on providing clear and step-by-step visual representations of algorithms, enabling students to understand complex concepts more effectively. It primarily covers

fundamental topics such as sorting algorithms, searching techniques, tree traversals, and basic graph algorithms, which are essential for academic learning and technical interviews.

The system is designed to offer real-time animations synchronized with pseudocode, allowing users to observe how each step of an algorithm is executed. It also includes interactive controls such as play, pause, step forward, and speed adjustment to support self-paced learning. The application works offline, ensuring accessibility for students without continuous internet connectivity, and is optimized for Android devices, making it portable and easy to use anytime and anywhere.

Additionally, the project follows a modular architecture, which allows future expansion by adding more advanced algorithms, features, or enhancements such as performance analysis or intelligent assistance. However, the current scope is limited to visualization and understanding of algorithms and does not include full-scale coding environments, advanced AI-based tutoring, or support for all complex algorithms. Overall, the project aims to provide a simple, scalable, and user-friendly platform for improving algorithm learning through visualization.

Chapter 5

Proposed System

5. Proposed System

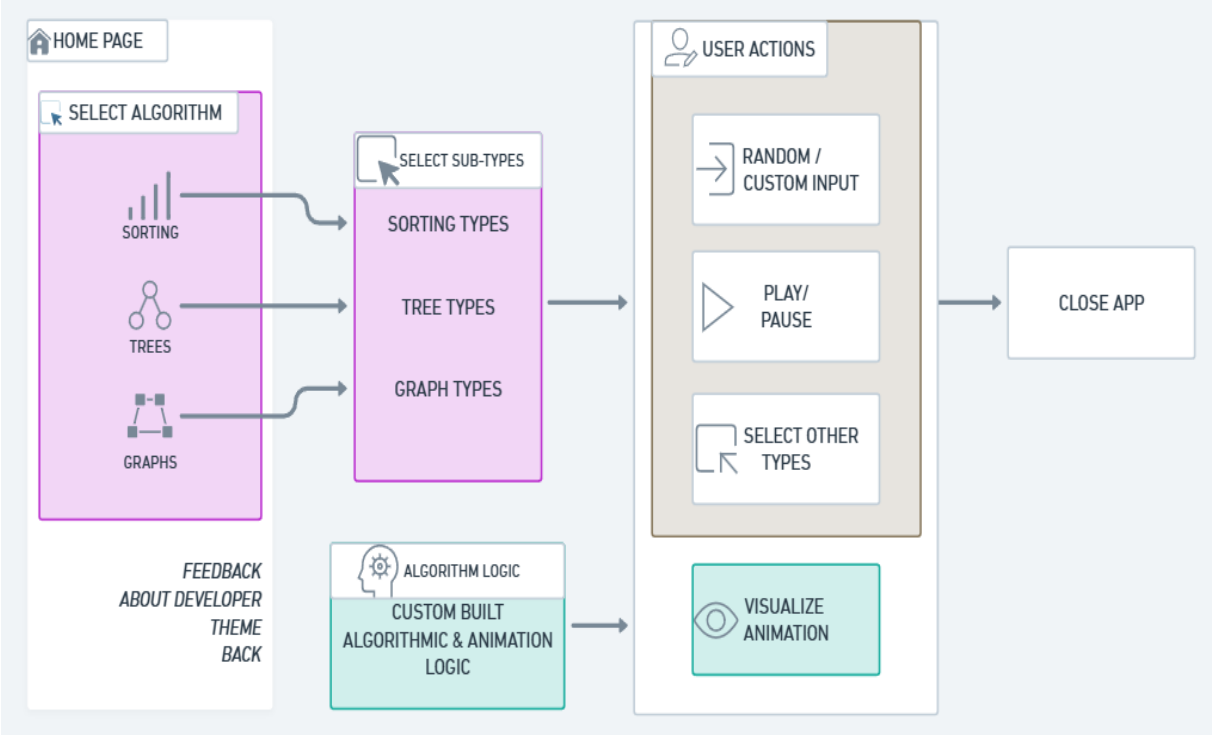


Figure 5.1: Proposed System

This diagram represents the overall architecture and workflow of the proposed AlgoPilot system. The process begins at the Home Page, where the user selects an algorithm category such as sorting, trees, or graphs. Based on this selection, the system displays corresponding sub-types, allowing the user to choose a specific algorithm for visualization. Once the algorithm is selected, the user can perform actions such as providing custom or random input, controlling execution using play/pause options, or switching between different algorithm types.

The selected input is then processed by the Algorithm Logic module, which consists of custom-built algorithmic and animation logic. This module ensures that each step of the algorithm is executed accurately and is synchronized with visual representation. The processed data is passed to the Visualization module, where animations are generated to illustrate the step-by-step execution of the algorithm.

The system is designed to be interactive and user-friendly, enabling learners to control and observe algorithm behavior in real time. Additional features like feedback, theme settings, and navigation options enhance usability. Overall, the architecture highlights a modular design where user interaction, algorithm processing, and visualization work together to provide an effective learning experience.

1. Data Flow Diagram (DFD)

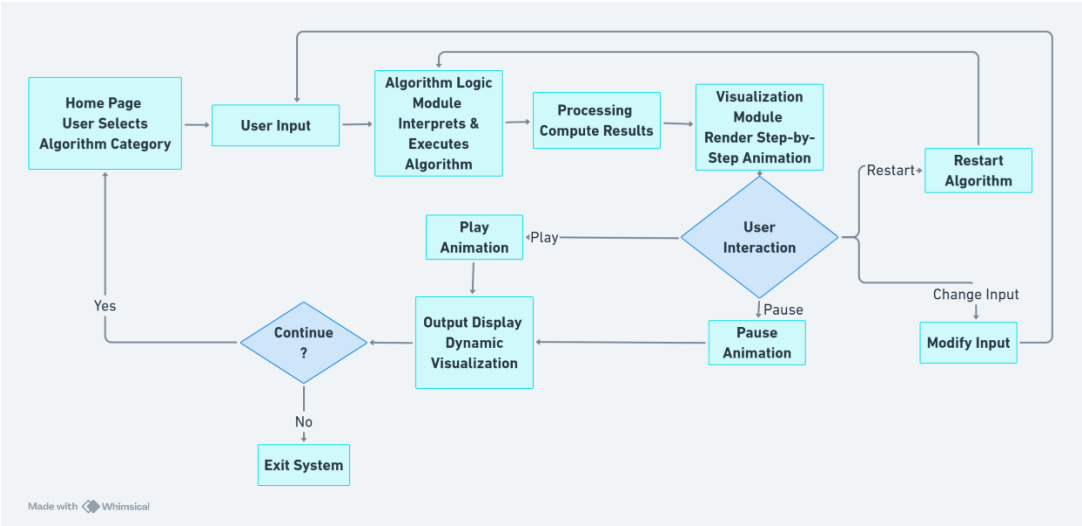


Figure 5.1.1: Data Flow Diagram

The Data Flow Diagram represents the logical flow of data within the Algorithm Visualization System. It outlines how user inputs are processed and transformed into meaningful visual outputs.

The process begins on the Home Page, where the user selects a specific algorithm category. This selection is passed to the Algorithm Logic Module, which interprets the input and executes

the corresponding algorithm using pre-defined logic. The computed results are then transmitted to the Visualization Module, responsible for rendering dynamic, step-by-step animations of the algorithm in execution.

Throughout this process, the user can interact with the system through actions such as Play, Pause, Restart, or Change Input, thereby enhancing engagement and learning. Once the visualization is complete, the user can terminate or return to the main interface.

Flow Summary:

User Input → Algorithm Logic Module → Processing → Visualization Module → User Interaction → Output Display

2. Use Case Diagram

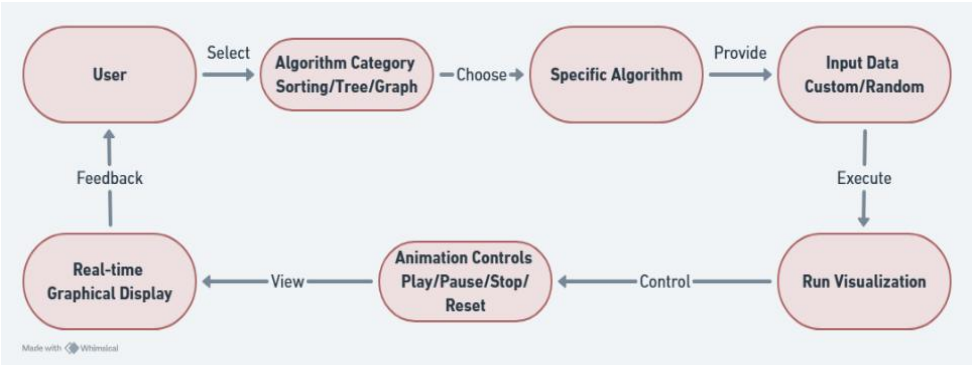


Figure 5.2.1: Use Case Diagram

The Use Case Diagram depicts the relationship between the User and the Algorithm Visualization System, highlighting the functional aspects and possible interactions.

Primary Actor: User

System: Algorithm Visualization Application

Key Use Cases:

- Select algorithm category (e.g., Sorting, Tree, Graph)
- Choose a specific algorithm subtype
- Provide input data (custom or randomly generated)
- Execute algorithm visualization
- Control animation flow (Play, Pause, Stop, Reset)
- View real-time graphical representation of the algorithm

This diagram emphasizes how users actively interact with the application to gain a clear understanding of algorithmic behavior through visual learning.

3. Sequence Diagram

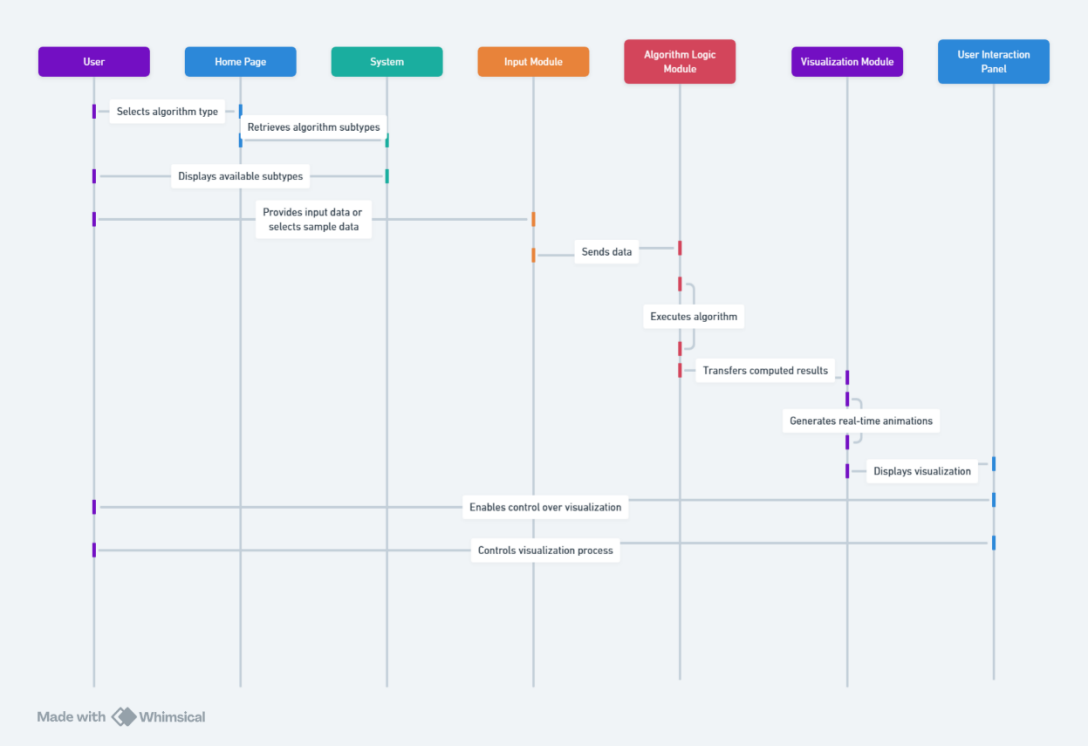


Figure 5.3.1: Sequence diagram

The Sequence Diagram presents a chronological representation of the interactions among system components during algorithm visualization.

Stepwise Flow:

- The user selects the algorithm type from the Home Page interface.
- The system retrieves and displays the available subtypes related to the chosen category.
- The user provides input data or opts for system-generated sample data.
- The Algorithm Logic Module executes the selected algorithm and processes the data.
- The computed results are transferred to the Visualization Module, which generates real-time graphical animations.
- The User Interaction Panel enables control over the visualization process, ensuring a seamless user experience.

This diagram outlines the sequential coordination between user actions and internal system operations, ensuring clarity in workflow and modular design.

4. Activity Diagram

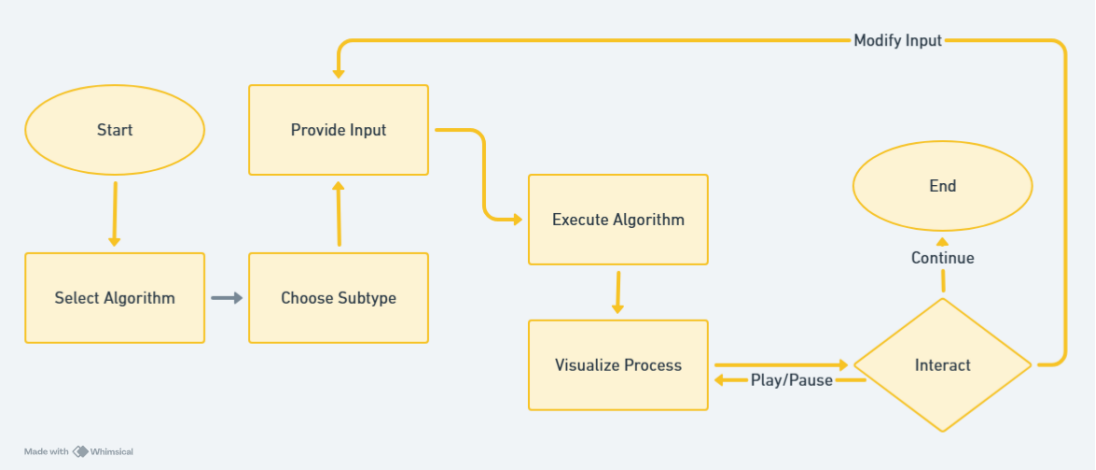


Figure 5.4.1: Activity diagram

The Activity Diagram provides a visual representation of the system's operational flow from initialization to completion.

Workflow:

Start → Select Algorithm → Choose Subtype → Provide Input → Execute Algorithm → Visualize Process → Interact (Play/Pause/Modify Input) → End

This diagram captures the complete user journey and system responses at each stage, ensuring that every process follows a structured and logical path.

5. Algorithm Workflow

The algorithmic process of the application follows a systematic and modular sequence:

1. **Start:** Initialize the system and load available algorithm categories.
2. **Select Category:** The user selects a major algorithm type (Sorting, Tree, or Graph).
3. **Choose Subtype:** The system presents subtypes (e.g., Quick Sort, Merge Sort).
4. **Input Data:** User enters custom data or allows automatic data generation.
5. **Execute Logic:** The Algorithm Logic Module applies the relevant algorithm to process input data.
6. **Generate Animation:** The Animation Engine visualizes each computational step in real-time.
7. **User Interaction:** The user may play, pause, or modify inputs during visualization.
8. **Display Result:** The system presents the final outcome and awaits user termination or navigation back to the home screen.

6. Key Components of the Proposed Framework

- **Home Page Interface:** Serves as the entry point of the system, allowing users to navigate between different algorithm categories and subtypes through an intuitive layout.
- **Algorithm Logic Module:** Implements algorithm-specific logic using optimized data structures. It ensures accurate computation and forms the foundation for animation generation.
- **Animation Engine:** Transforms algorithmic computations into dynamic, step-by-step animations. It provides visual feedback that helps users understand algorithm behaviour intuitively.
- **User Interaction Panel:** Offers control elements such as Play, Pause and Change Input for interactive visualization. Enhances learning by allowing users to explore algorithms at their own pace.
- **Feedback Section:** Provides a channel for users to share experiences, suggest improvements, and provide developer feedback, promoting iterative refinement of the application.

7. System Architecture

The system architecture of AlgoPilot is designed in a modular manner, consisting of three main layers:

- **User Interface Layer:** Handles user interactions and displays visualizations.
- **Application Logic Layer:** Implements algorithms and controls animation flow.
- **Data Layer:** Manages input data and algorithm states.
- **Scope:** To organize the system into structured components.
- **Use Case:** Ensures smooth communication between UI and algorithm logic.
- **Focus:** Modularity, scalability, and efficient data handling.

Chapter 6

Experimental Setup

6. Experimental Setup

1. Development Environment

IDE: Android Studio

Version Control: Git / GitHub

Testing Tools: Android Emulator

- **Scope:** To maintain a collaborative, version-controlled development process with continuous testing.
- **Use Case**: Team collaboration, tracking code changes, and evaluating performance metrics.
- **Focus:** Code maintainability, performance monitoring, and reliable testing workflows.

2. Platform

Android Studio: Android Studio serves as the core development platform for building and testing the application. It provides powerful tools for code editing, debugging, UI design, and performance profiling, ensuring a streamlined development workflow.

- **Scope:** To enable end-to-end Android app development and testing within a unified environment.
- **Use Case:** Developers use Android Studio to implement algorithm visualizations, simulate animations, and test app behavior across multiple deivces.
- **Focus:** Efficiency in development, debugging accuracy, and performance optimization for real-time animations.

3. Language

Java: Java is the primary programming language used to implement the app's logic, algorithm animations, and user interaction layers. It ensures high compatibility with Android systems and supports structured, object-oriented programming for maintainable code.

- **Scope:** To handle algorithmic operations, visual logic simulations, and app navigation efficiently.
- **Use Case:** Defining algorithm workflows (like sorting or searching), rendering visual representations, and managing UI components.
- **Focus:** Stability, reusability, and seamless user interaction through efficient event handling.

4. Build Tool

Gradle: Gradle automates the build process, testing, and packaging of the Android app. It manages dependencies and external libraries needed for animations, visualizations, and advanced UI elements.

- **Scope:** To ensure consistent, automated builds and dependency management.
- **Use Case:** Integrating third-party libraries for animations, UI enhancement, and version control during development.
- **Focus:** Build automation, dependency resolution, and smooth integration of external modules.

5. Software Requirements

- Android SDK (latest stable version): Provides the necessary APIs and tools for Android development.
- JDK 8 or higher: Ensures compatibility and optimal performance for Javabased Android applications.

- Emulator or Physical Android Device: Used to test responsiveness, performance, and user experience across devices.
    - **Scope:** Establish a stable foundation for app development and execution.
    - **Use Case:** Testing app compatibility, responsiveness, and visual performance.
    - **Focus:** Reliability, stability, and compatibility across environments.

6. Operating Environment

The app is designed to operate smoothly across various Android devices and screen resolutions, ensuring accessibility and usability.

- **Scope:** Deliver a responsive, user-friendly learning experience even without internet connectivity.
- **Use Case:** Enables students and learners to visualize algorithms in offline mode for uninterrupted study sessions.
- **Focus:** Offline functionality, responsiveness, and optimized real-time animation performance.

Chapter 7

Results and Discussion

7. Results and Discussion

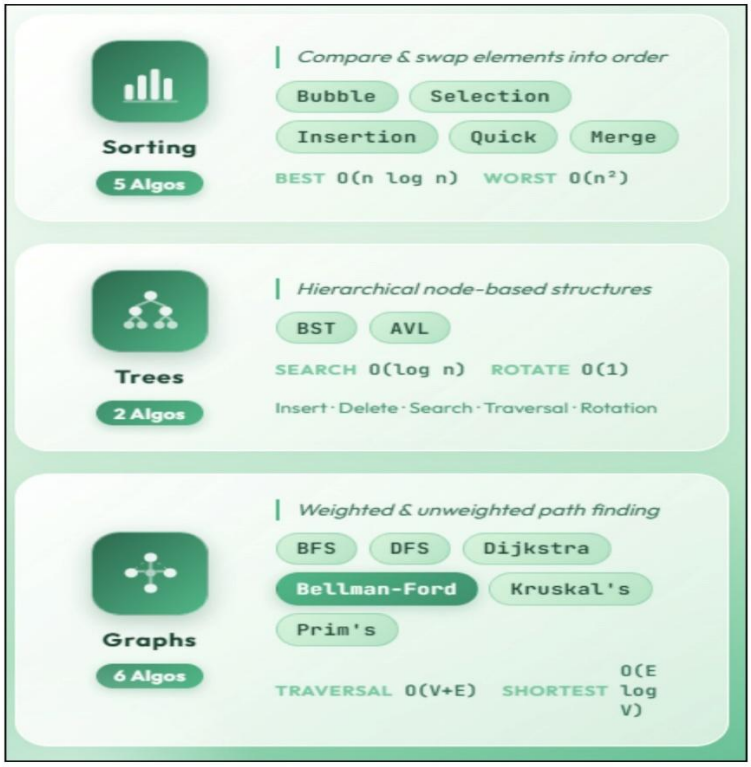


Figure 7.1: Algorithms Implemented

1. Sorting Algorithms

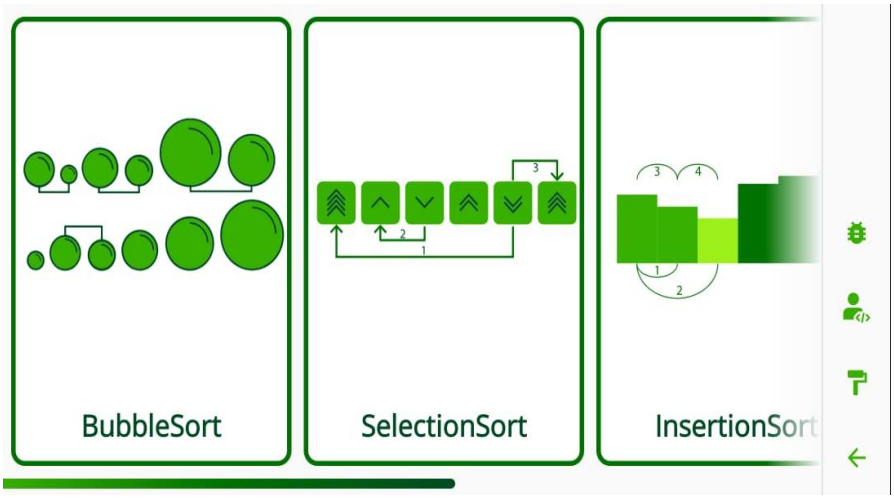


Figure 7.2: Sorting Homepage

This figure represents the sorting homepage of the AlgoPilot application, where users can select different sorting algorithms for visualization. The interface displays options such as Bubble Sort, Selection Sort, and Insertion Sort in a card-based layout. Each card includes a visual icon that gives a basic idea of how the algorithm operates. The design is simple and user-friendly, allowing easy navigation between different sorting techniques. Users can scroll horizontally to explore additional algorithms if available. The green color theme enhances visual clarity and maintains consistency throughout the application. This page acts as the entry point for learning sorting algorithms through interactive visualization. Overall, it provides an intuitive and engaging way for users to begin exploring sorting concepts.

1.1 Bubble Sort

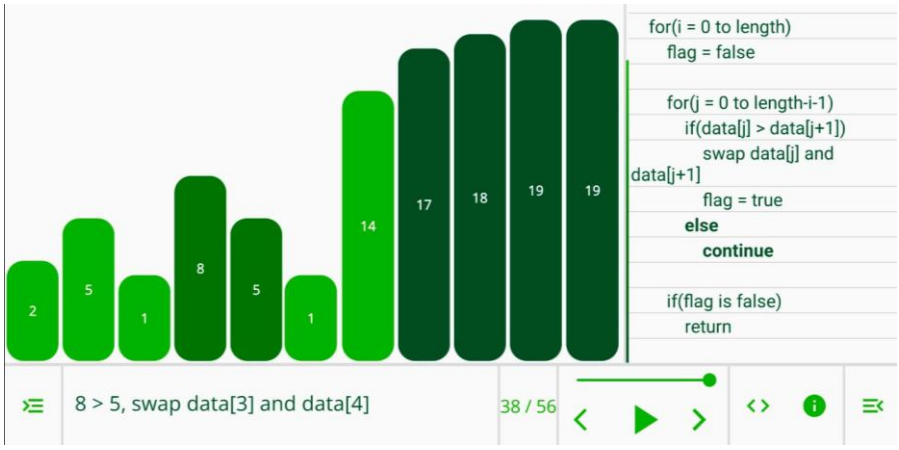


Figure 7.3: Bubble Sort

Bubble Sort is a simple comparison-based sorting algorithm where adjacent elements are repeatedly compared and swapped if they are in the wrong order. After each pass, the largest unsorted element “bubbles up” to its correct position at the end of the array. This process continues until the entire array is sorted.

Working Principle:

- Compare adjacent elements: $data[j]$and $data[j+1]$
- If $data[j] > data[j+1]$, swap them
- Repeat for all elements in each pass
- Stop early if no swaps occur (optimized case)

Mathematical Representation:

- Number of comparisons in worst case:

$$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$$

Key Features:

- Simple and easy to understand
- Stable sorting algorithm
- Inefficient for large datasets

1.2 Selection Sort

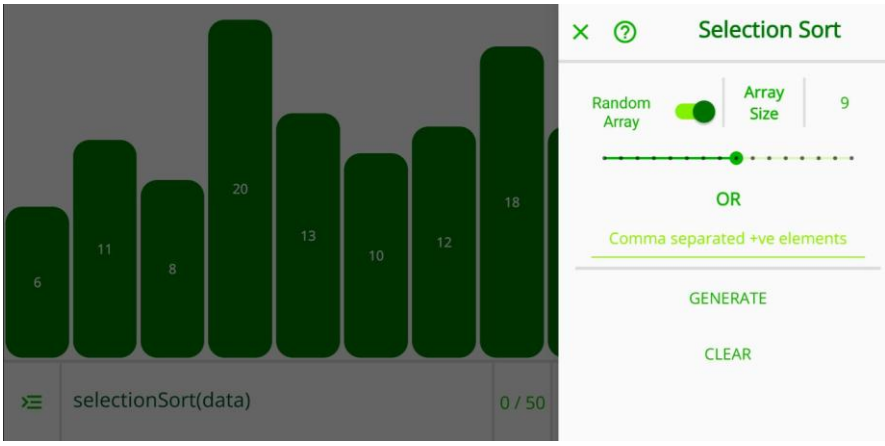


Figure 7.4: Selection Sort

Selection Sort works by repeatedly selecting the minimum element from the unsorted portion of the array and placing it at the beginning. Unlike Bubble Sort, it minimizes the number of swaps by performing only one swap per iteration.

Working Principle:

- Assume the first element is the minimum
- Compare it with remaining elements
- Find the smallest element
- Swap it with the first element

- Move to the next position and repeat

Mathematical Representation:

- Number of comparisons:

$$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$$

Key Features:

- Performs fewer swaps compared to Bubble Sort
- Not stable by default
- Simple but not efficient for large inputs

1.3 Insertion Sort

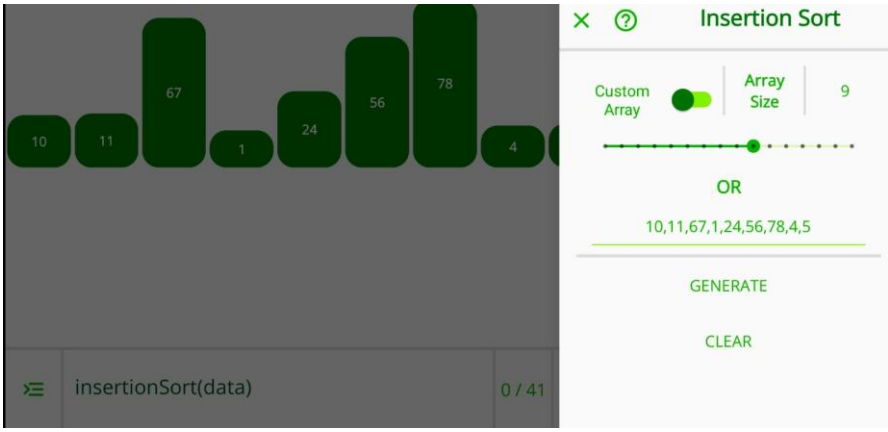


Figure 7.5: Insertion Sort

Insertion Sort is a simple and efficient sorting algorithm that works similarly to the way we sort playing cards in our hands. It builds the final sorted array one element at a time by repeatedly inserting each element into its correct position in the already sorted portion of the array.

Working Principle:

- Start from the second element of the array
- Compare the current element with the previous elements
- Shift all larger elements one position to the right
- Insert the current element into its correct position
- Repeat the process until the entire array is sorted

Mathematical Representation:

- Number of comparisons in worst case:

$$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$$

Key Features:

- Efficient for small datasets
- Performs well for nearly sorted arrays
- Stable sorting algorithm (maintains relative order)
- Simple to implement and understand

1.4 Quick Sort

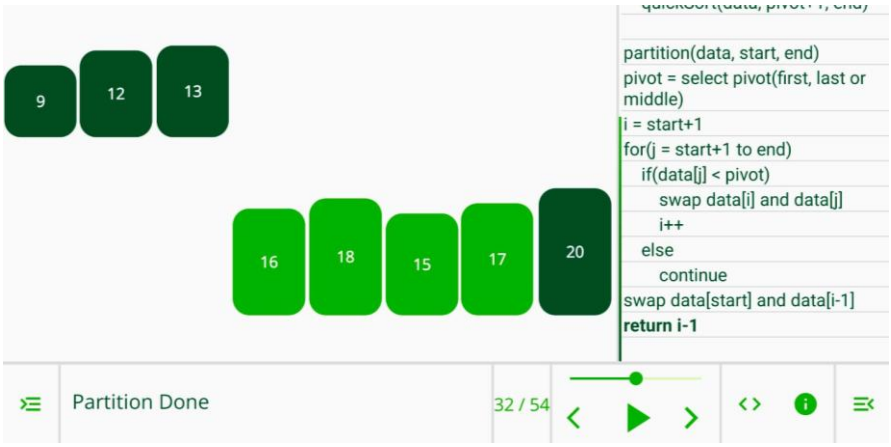


Figure 7.6: Quick Sort

Quick Sort is a highly efficient divide-and-conquer sorting algorithm that works by selecting a pivot element and partitioning the array into two subarrays: elements less than the pivot and elements greater than the pivot. The same process is then recursively applied to the subarrays until the entire array is sorted.

Working Principle:

- Select a pivot element (first, last, or middle)
- Partition the array into two parts:
  - Left → elements < pivot
  - Right → elements > pivot
- Recursively apply Quick Sort on both subarrays
- Combine the results to get a sorted array

Mathematical Representation:

- Recurrence relation:

$$T(n) = T(k) + T(n - k - 1) + O(n)$$

Key Features:

- Fastest sorting algorithm in practice
- Uses divide-and-conquer strategy
- In-place sorting (no extra memory required)
- Not stable

1.5 Merge Sort

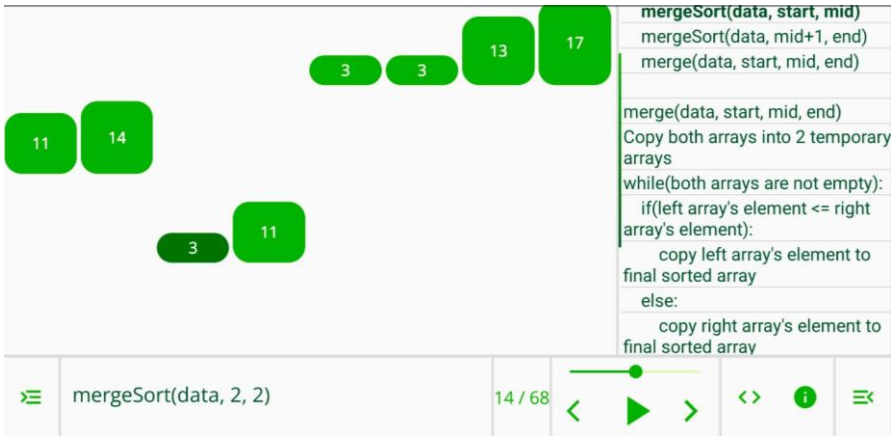


Figure 7.7: Merge Sort

Merge Sort is a divide-and-conquer algorithm that divides the array into smaller subarrays, sorts them recursively, and then merges them back together in a sorted manner. It guarantees consistent performance regardless of input order.

Working Principle:

- Divide the array into two halves
- Recursively sort each half
- Merge the sorted halves into a single sorted array
- Repeat until the entire array is sorted

Mathematical Representation:

- Recurrence relation:

$$T(n) = 2T(n/2) + O(n)$$

- Solving using Master Theorem:

$$T(n) = O(\text{nlog}\, n)$$

Key Features:

- Stable sorting algorithm

- Consistent performance in all cases
- Uses divide-and-conquer approach

2. Tree Algorithms

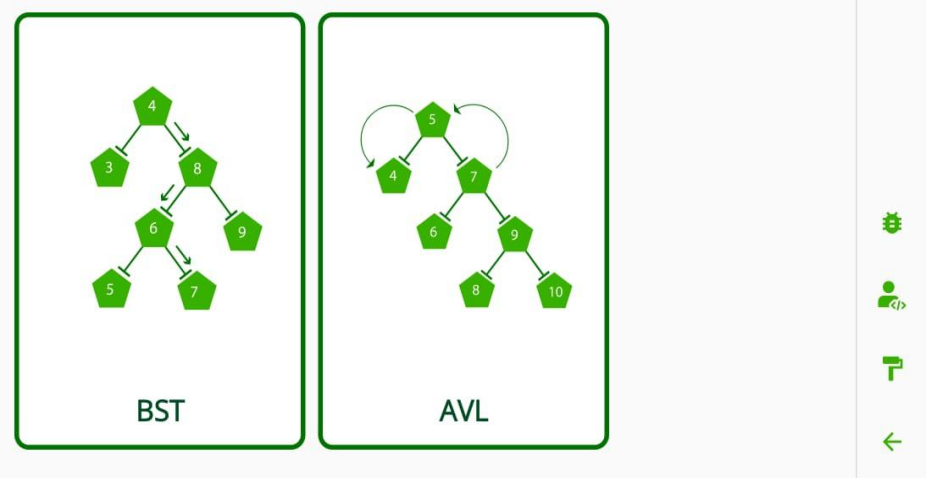


Figure 7.8: Tree Homepage

This figure represents the tree data structure homepage of the AlgoPilot application, where users can choose between different tree types for visualization. The interface displays options such as Binary Search Tree (BST) and AVL Tree in a structured, card-based layout. Each card features a diagram that visually demonstrates the structural properties and balancing mechanics of the specific tree. While the BST card shows a potentially unbalanced or "skewed" hierarchy, the AVL card highlights the self-balancing nature using rotation indicators. The clean, green-themed design ensures visual clarity and maintains a consistent aesthetic across the learning platform. This page serves as the primary entry point for users to interactively explore how data is organized and searched within hierarchical structures. By providing side-by-side comparisons, the interface intuitively teaches the importance of tree height and balance in computational efficiency.

2.1 AVL Tree

An AVL Tree is a self-balancing Binary Search Tree where the difference in height between the left and right subtrees of any node is at most 1. This balance is maintained automatically after every insertion or deletion using rotation operations, ensuring efficient performance.

Balance Factor:

$$BF = \text{Height of Left Subtree} - \text{Height of Right Subtree}$$

$$BF \in \{-1, 0, +1\}$$

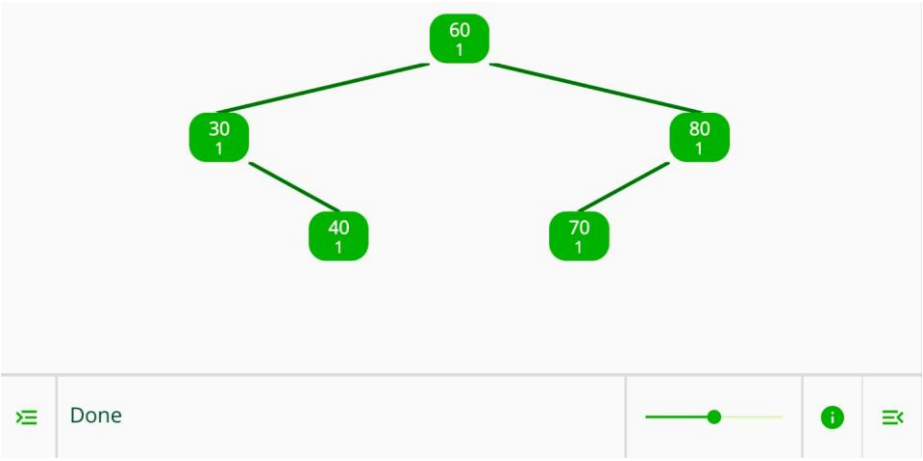


Figure 7.9: AVL Tree

In Figure 7.9, the AVL tree is shown in a balanced state. The root node (60) has two subtrees, and the height difference between the left and right subtrees is within the allowed range. All nodes satisfy the AVL property where the balance factor is either -1, 0, or +1. This ensures that the tree remains height-balanced and operations such as search can be performed efficiently.

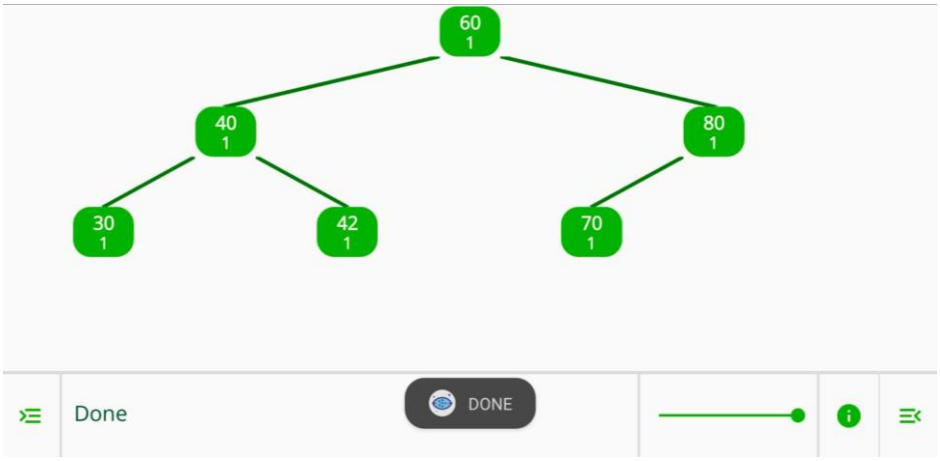


Figure 7.10: Insertion in AVL tree

In Figure 7.10, a new node is inserted into the AVL tree. After insertion, the structure of the tree changes, and the system checks the balance factor of each node. If the tree becomes unbalanced, appropriate rotations (such as left rotation, right rotation, or a combination of both) are applied to restore balance. The figure demonstrates how the AVL tree reorganizes itself after insertion while maintaining the Binary Search Tree property.

Key Features:

- Self-balancing Binary Search Tree
- Maintains balance factor $(-1,0,+1)$
- Uses rotations (LL, RR, LR, RL) to stay balanced
- Ensures height = $O(\log\ n)$
- Provides fast search, insertion, deletion

- Prevents tree from becoming skewed
- Suitable for applications requiring frequent searching

2.2 Binary Search Tree(BST)

A Binary Search Tree (BST) is a hierarchical data structure in which each node follows the property:

- All elements in the left subtree are less than the root node
- All elements in the right subtree are greater than the root node

This property is recursively followed for all nodes in the tree.

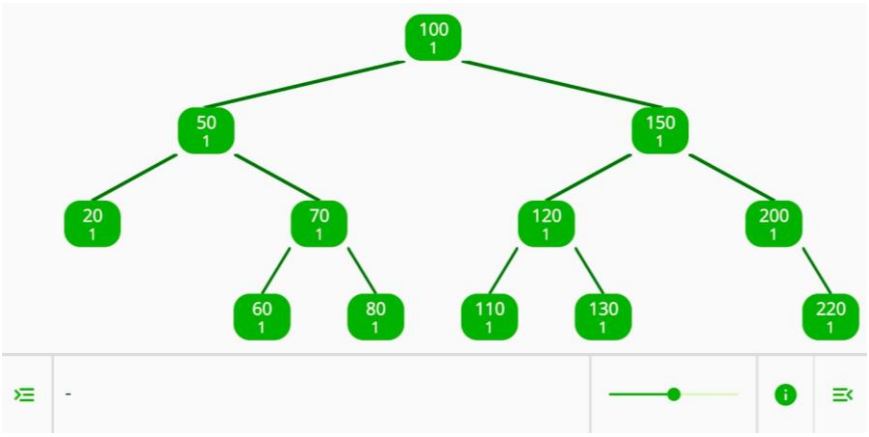


Figure 7.11: BST

In Figure 7.11, the BST structure is shown where the root node is **100**. All nodes on the left side (such as 50, 20, 70, etc.) are smaller than 100, while nodes on the right side (such as 150, 120, 200, etc.) are greater. This clearly demonstrates the BST property and how data is organized in a sorted hierarchical manner.

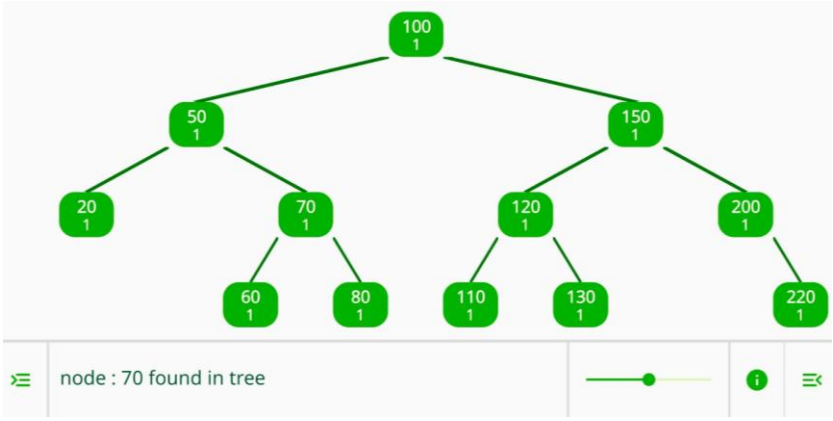


Figure 7.12: Searching in BST

In Figure 7.12, the searching operation in a BST is illustrated. To search for a value (e.g., 70), the process starts from the root node:

- Compare the target value with the root (100) → since 70 < 100, move to the left subtree

- Compare with node 50 → since 70 > 50, move to the right subtree
- Compare with node 70 → element found

This step-by-step comparison reduces the search space, making the operation efficient.

Key Features:

- Hierarchical tree structure
- Left subtree values < root < right subtree values
- Efficient searching and sorting
- Average time complexity: $O(\log\ n)$
- Can become unbalanced (skewed) → worst case $O(n)$
- Simple and easy to implement
- Used as a base for advanced trees like AVL

3. Graph

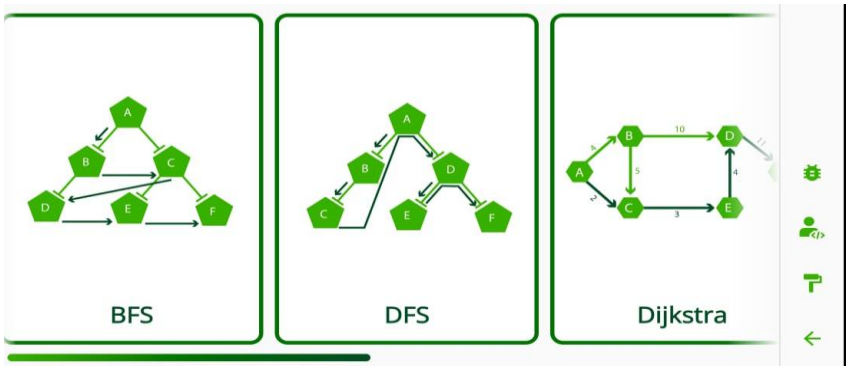


Figure 7.13: Graph Homepage

This figure represents the graph homepage of the AlgoPilot application, where users can select various graph traversal and pathfinding algorithms for interactive visualization. The interface features a card-based layout showcasing popular techniques like Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra's Algorithm. Each card includes a specialized diagram that illustrates the unique movement patterns of these algorithms, such as the level-by-level approach of BFS versus the deep-branch exploration of DFS. The Dijkstra card specifically displays a weighted graph, highlighting its role in finding the shortest path between nodes. The consistent green color scheme and intuitive icons provide a clean, modern aesthetic that simplifies complex computer science concepts. This page serves as a central hub, allowing students and developers to navigate easily between different graph-based logic scenarios. Overall, the design offers an engaging entry point for mastering the foundational principles of graph theory through visual feedback.

3.1 Breadth First Search (BFS)

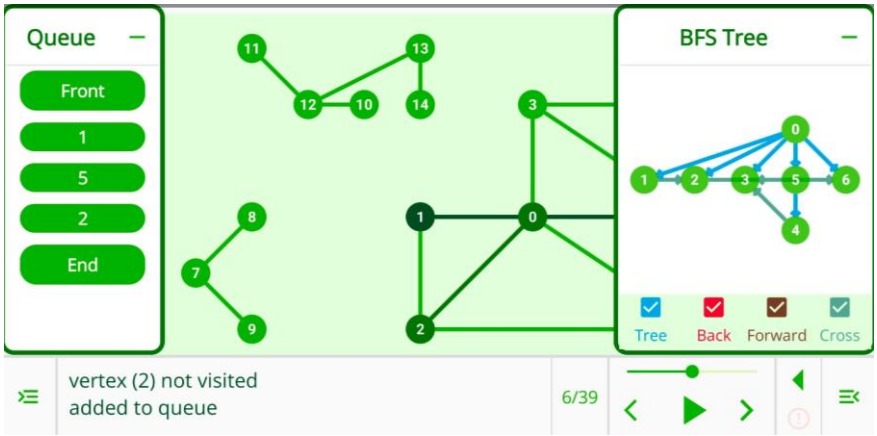


Figure 7.14: BFS

Breadth First Search (BFS) is a graph traversal algorithm that explores vertices level by level starting from a source node. It uses a queue data structure to keep track of nodes that need to be visited.

In the figure, the algorithm starts from a source vertex and visits all its adjacent vertices first. These vertices are added to the queue. Then, one by one, vertices are removed from the front of the queue and their unvisited neighbors are explored. The BFS tree shown represents the order in which nodes are visited.

Working Principle:

- Start from source node
- Mark it as visited
- Insert into queue
- Repeat until queue is empty:
  - Remove front node
  - Visit all unvisited adjacent nodes
  - Add them to queue

Mathematical Representation:

- **Time Complexity:**

$$O(V + E)$$

Where:

V= number of vertices

E= number of edges

- **Space Complexity:**

$$O(V)$$

Key Features:

- Traverses graph level by level
- Uses Queue (FIFO)
- Finds shortest path in unweighted graph
- Visits all nodes systematically
- Useful for network traversal & connectivity

3.2 Depth First Search (DFS)

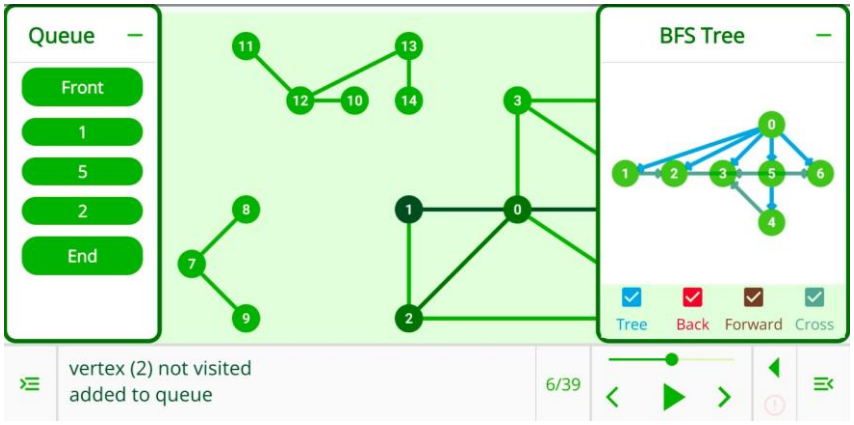


Figure 7.15: DFS

Depth First Search (DFS) is a graph traversal algorithm that explores nodes as deep as possible before backtracking. It uses a stack (or recursion) to keep track of vertices.

In the figure, the traversal starts from a node and moves along one branch until it reaches the deepest node. If no further nodes are available, it backtracks and explores other branches. The DFS tree shows this deep traversal structure.

Working Principle:

- Start from source node
- Mark it as visited
- Visit one adjacent node recursively
- Continue until no unvisited nodes remain
- Backtrack and explore remaining nodes

Mathematical Representation:

- **Time Complexity:**

$$O(V + E)$$

- **Space Complexity:**

$$O(V)$$

Key Features:

- Traverses graph depth-wise
- Uses Stack / Recursion
- Explores one path fully before backtracking
- Useful for cycle detection & path finding

3.3 Dijkstra's Algorithm

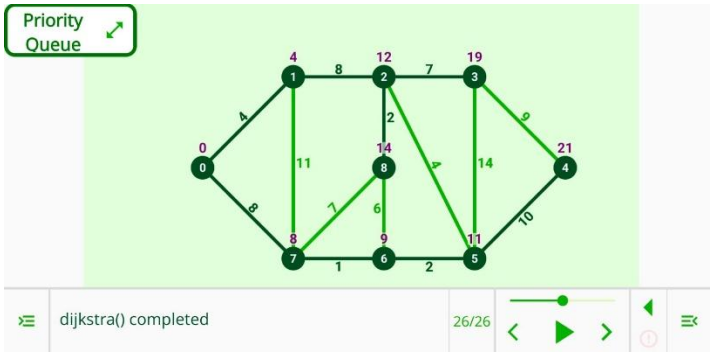


Figure 7.16: Dijkstras Algorithm

Dijkstra's Algorithm is used to find the shortest path from a source node to all other nodes in a weighted graph (with non-negative weights). It uses a priority queue to always expand the node with the smallest distance.

In the figure, nodes are assigned distance values from the source. Initially, the source node has distance 0, and all others are infinity. The algorithm repeatedly selects the node with the minimum distance and updates its neighboring nodes. This process continues until all nodes have been visited.

Working Principle:

- Assign distance 0 to source, ∞ to others
- Use priority queue
- Select node with minimum distance
- Update distances of adjacent nodes:

$$\text{if } d[u] + w(u, v) < d[v], \text{ then } d[v] = d[u] + w(u, v)$$

- Repeat until all nodes are processed

Mathematical Representation:

- **Time Complexity:**

$$O((V+E)\log V)$$

- **Space Complexity:**

$$O(V)$$

Key Features:

- Finds shortest path in weighted graph
- Works only with non-negative weights
- Uses Priority Queue
- Greedy approach (chooses minimum distance node)
- Time Complexity: $O((V+E)\log V)$
- Used in GPS and routing systems

3.4 Bellman Ford Algorithm

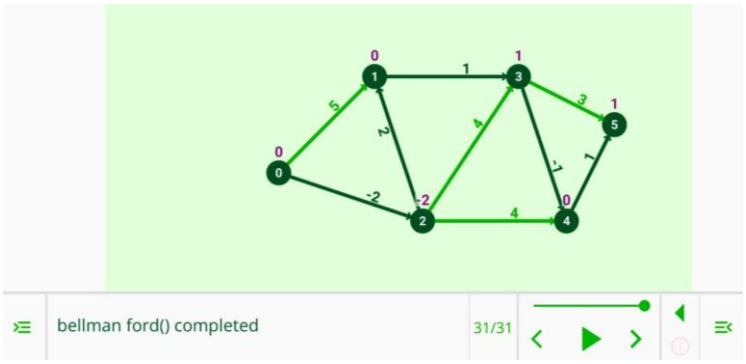


Figure 7.17: Bellman Ford Algorithm

Bellman-Ford Algorithm is used to find the shortest path from a single source vertex to all other vertices in a weighted graph. Unlike Dijkstra's algorithm, it can handle negative edge weights.

In the figure, the source node starts with distance 0, while all other nodes are initialized to infinity. The algorithm relaxes all edges repeatedly for $V-1$iterations. After each iteration, the shortest distances are updated. The figure shows how distances are progressively minimized until the final shortest paths are obtained.

Working Principle:

- Initialize distances:

$$d[source]=0, d[v]=\infty$$

- Relax all edges $V-1$times

- For each edge (u, v):

$$\text{if } d[u] + w(u, v) < d[v], d[v] = d[u] + w(u, v)$$

- Check for negative weight cycles:

If further relaxation is possible → cycle exists

Mathematical Representation:

- **Time Complexity:**

$$O(V \cdot E)$$

- **Space Complexity:**

$$O(V)$$

Key Features:

- Finds shortest path with negative weights
- Can detect negative weight cycles
- Relaxes edges $V - 1$times
- More flexible but slower than Dijkstra

3.5 Kruskal's Algorithm

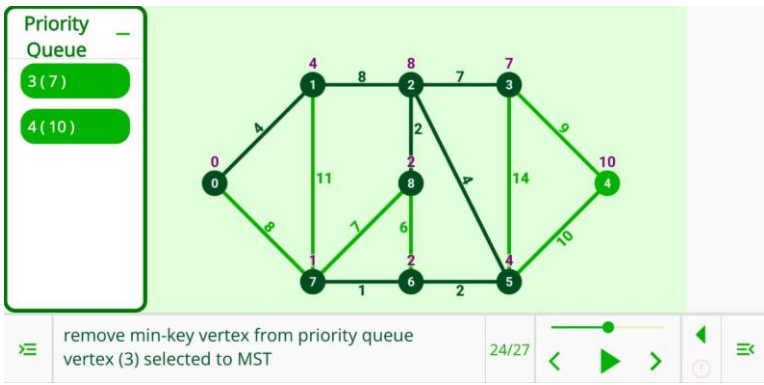


Figure 7.18: Kruskal's algorithm

Kruskal's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a graph. It selects edges in increasing order of weight while avoiding cycles.

In the figure, edges are arranged in a priority queue based on their weights. The algorithm repeatedly picks the smallest edge and adds it to the MST if it does not form a cycle. The process continues until all vertices are connected.

Working Principle:

- Sort edges in increasing order
- Initialize each vertex as a separate set
- Select smallest edge
- Check cycle using Union-Find:
  - If no cycle → include edge
  - Else → discard edge
- Repeat until MST is formed

Mathematical Representation:

- **Time Complexity:**

$$O(E\log\ E)$$

- **Space Complexity:**

$$O(V)$$

Key Features:

- Finds Minimum Spanning Tree (MST)
- Uses Greedy approach
- Selects edges in increasing order of weight
- Uses Union-Find to avoid cycles
- Time Complexity: $O(E\log\ E)$
- Works well for sparse graphs

3.6 Prim's Algorithm

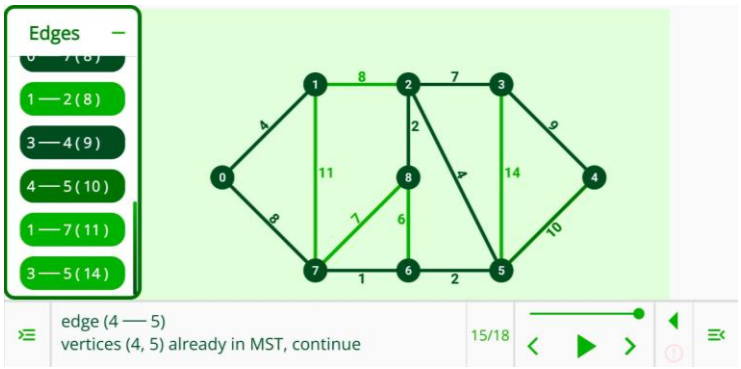


Figure 7.19: Prim's Algorithm

Prim's Algorithm is another greedy algorithm used to find the Minimum Spanning Tree (MST). It starts from a node and grows the MST by adding the smallest edge connecting the tree to a new vertex.

In the figure, the algorithm begins from a source node and repeatedly selects the minimum weight edge connecting visited and unvisited vertices. The edges selected are shown step-by-step, forming the MST.

Working Principle:

- Start with any vertex
- Initialize set of visited nodes
- Select minimum weight edge connecting visited to unvisited node
- Add edge to MST
- Repeat until all vertices are included

Mathematical Representation:

- **Time Complexity:**

$$O((V + E)\log V)$$

- **Space Complexity:**

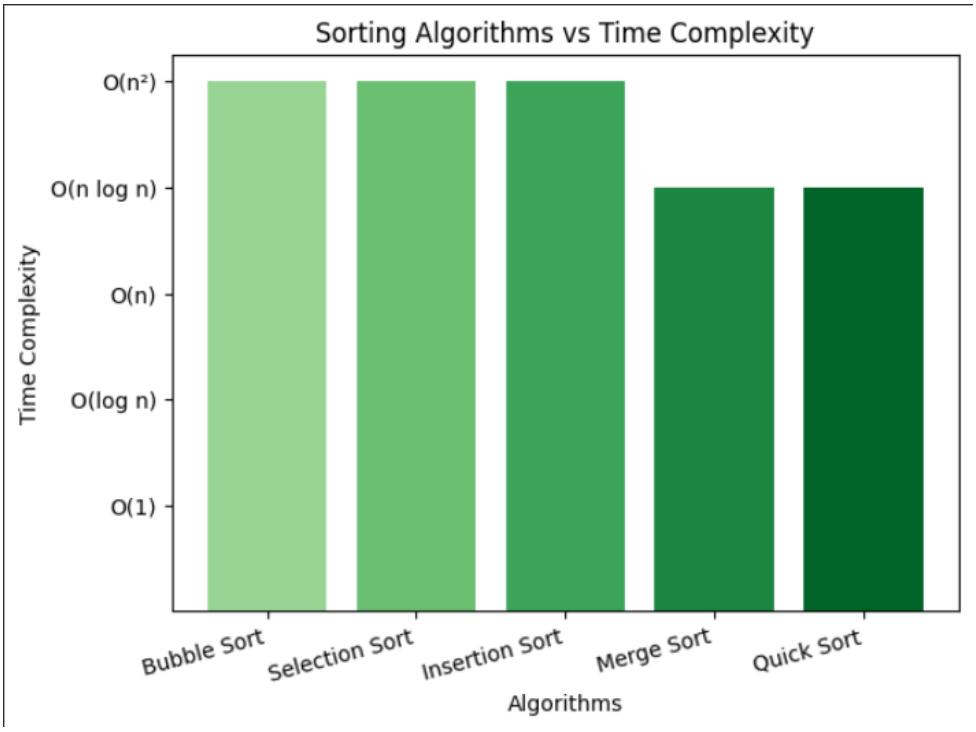
$$O(V)$$

Key Features:

- Finds Minimum Spanning Tree (MST)
- Starts from a node and expands tree
- Uses Priority Queue
- Always selects minimum edge
- Time Complexity: $O((V + E)\log V)$
- Works well for dense graphs

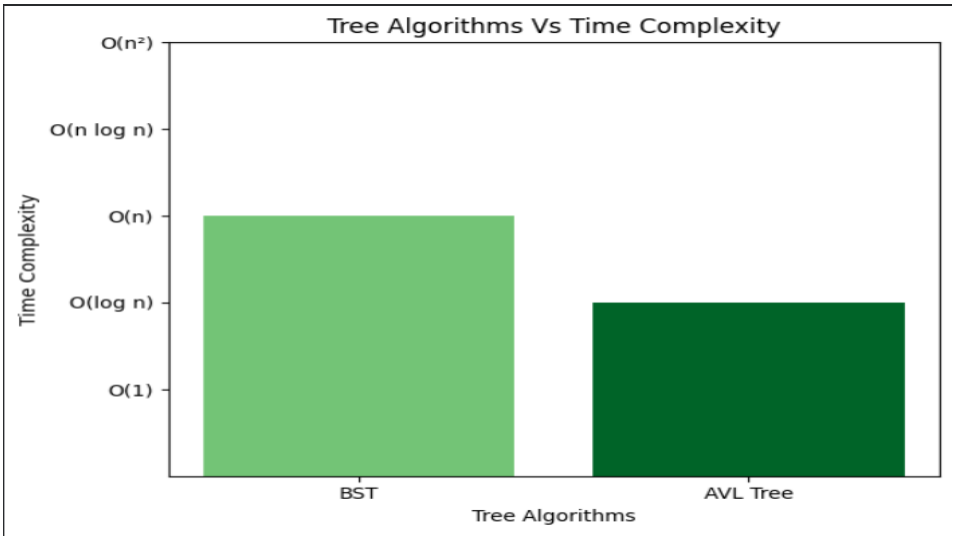
Table 7.1: Performance Analysis

Algorithm	Time	Accuracy
Bubble Sort	Animation Speed + O(n^2)	100%
Selection Sort	Animation Speed + O(n^2)	100%
Insertion Sort	Animation Speed + O(n^2)	100%
Merge Sort	Animation Speed + O(n log n)	100**%**
Quick Sort	Animation Speed + O(n log n)	100%
Tree Traversal	Animation Speed + O(n)	100%
BFS (Graph Traversal)	Animation Speed + O(V+E)	100%
DFS (Graph Traversal)	Animation Speed + O(V+E)	100%



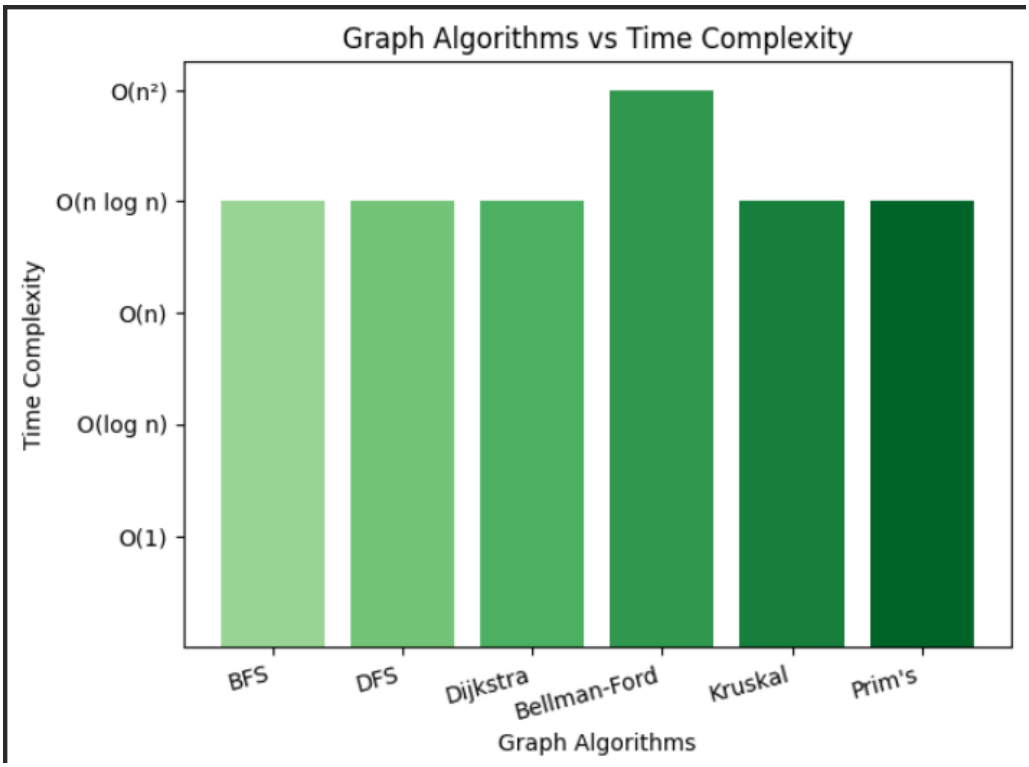
Graph 7.1: Sorting Algorithms Vs Time Complexity

This graph compares the time complexity of different sorting algorithms based on their efficiency. Bubble Sort, Selection Sort, and Insertion Sort show a time complexity of $O(n^2)$, which means their execution time increases rapidly as the input size grows. These algorithms are simple to implement but are inefficient for large datasets. On the other hand, Merge Sort and Quick Sort have a better time complexity of $O(n\log\ n)$, making them more suitable for handling larger inputs. The graph clearly shows that algorithms with lower complexity perform better as the data size increases. It highlights the importance of choosing efficient algorithms for optimal performance. Overall, the comparison demonstrates that advanced sorting techniques significantly reduce execution time compared to basic methods.



Graph 7.2: Tree Algorithms Vs Time Complexity

This graph compares the time complexity of Binary Search Tree (BST) and AVL Tree operations. The BST shows an average time complexity of $O(n)$, which may degrade to linear performance if the tree becomes unbalanced. This happens when elements are inserted in a sorted order, causing the tree to become skewed. In contrast, the AVL Tree maintains a balanced structure using rotations, ensuring a time complexity of $O(\log\ n)$for operations like search, insertion, and deletion. The graph highlights that AVL Trees provide more efficient and consistent performance compared to BSTs. Although AVL Trees require extra operations for balancing, they significantly improve efficiency for large datasets. Overall, the comparison demonstrates the advantage of self-balancing trees in maintaining optimal performance.



Graph 7.3: Graph Algorithms Vs Time Complexity

This graph compares the time complexity of various graph algorithms used for traversal, shortest path, and minimum spanning tree problems. BFS and DFS have a time complexity of $O(V + E)$, making them efficient for exploring all vertices and edges in a graph. Dijkstra's algorithm also performs efficiently with a complexity of $O((V + E)\log\ V)$, especially for graphs with non-negative weights. Bellman-Ford, however, has a higher complexity of $O(VE)$, as it relaxes all edges multiple times, making it slower for large graphs. Kruskal's and Prim's algorithms, used for finding minimum spanning trees, both have complexities around $O(E\log\ E)$or $O((V + E)\log\ V)$. The graph clearly shows that Bellman-Ford is less efficient compared to other algorithms. Overall, it highlights how different graph algorithms vary in performance depending on their approach and use case.

Chapter 8

Conclusion and Future Work

8. Conclusion and Future Work

8.1 Conclusion

In this project, AlgoPilot has been developed as an Android-based mobile application to enhance the learning of Data Structures and Algorithms (DSA) through interactive visualization. The project addresses the challenges faced by students in understanding abstract algorithmic concepts by providing step-by-step animated visualizations synchronized with pseudocode. This approach helps learners clearly observe how algorithms operate in real time, making complex processes easier to understand.

The application successfully bridges the gap between theoretical knowledge and practical understanding by transforming static algorithm descriptions into dynamic visual representations. Features such as execution control (play, pause, step-by-step navigation, and speed adjustment) support self-paced learning, allowing users to analyze algorithm behavior in detail. Additionally, the application ensures offline accessibility and mobile compatibility, making it a portable and convenient learning tool for students.

Overall, AlgoPilot demonstrates that mobile-based visualization systems can significantly improve conceptual clarity, engagement, and accessibility in algorithm education. The project provides a user-friendly and scalable platform that can benefit students in academics, competitive programming, and technical interview preparation.

8.2 Future Scope

Although AlgoPilot provides an effective solution for algorithm visualization, there are several opportunities for further enhancement and expansion:

- **Addition of Advanced Algorithms:** Future versions can include more complex algorithms such as dynamic programming, backtracking, and advanced graph algorithms.
- **Integration of AI-Based Assistance:** An AI chatbot or intelligent tutor can be added to provide real-time explanations, guidance, and doubt-solving support.
- **Performance Analysis Features:** The application can be enhanced to display time and space complexity analysis, helping students compare algorithm efficiency.
- **User Custom Input Support:** Allowing users to input their own data and test algorithms on custom datasets can improve interactivity and learning depth.
- **Cross-Platform Development:** Expanding the application to other platforms such as iOS or web can increase accessibility for a wider audience.
- **Gamification and Quiz Features:** Adding quizzes, challenges, and rewards can make learning more engaging and motivate students to practice regularly.
- **Cloud Integration and Progress Tracking:** Future updates can include user accounts and cloud storage to track learning progress and performance.

References

[1] Bhandari, S. R., Abhijeet, C., et al., 2025. Algorithm visualization on data structures along with AI chatbot. IJIRCST, 13(1), 21–27.

[2] Matalia, Jeet & Kathiriand, Shreya & Patel, Priyanka. (2024). Demystifying Complex Algorithms: A Visual Guide Through the DSA Visualizer. 10.1007/978-981- 97-1313-4_18.

[3] Zhang, Yan. (2024). Application of Algorithm Visualization Techniques in Teaching Computer Data Structure Course. Applied Mathematics and Nonlinear Sciences. 9. 10.2478/amns-2024-0788.

[4] Kogan, G., Chassidim, H., & Rabaev, I. (2024). The efficacy of animation and visualization in teaching data structures: a case study. Educational technology research and development, 72(4), 2349 2372.

[5] Gupta, V., & Rao, D., 2023. Visualizing algorithms in the field of education: A comprehensive review. IJRASET, 11(7), 250–258.

[6] Lou, G., Sufang, W., & Shen, H. (2023, August). Research on Graphic Interactive Visual Online Teaching of Intelligent Algorithm Course. In EAI International Conference, BigIoT-EDU (pp. 506 516). Cham: Springer Nature Switzerland.

[7] Kumari, A., Bohra, N., & Sangwan, P. Evaluating Software Algorithm Visualization Projects.

[8] Liu, G. (2023, July). Research and Application of Animation Visualization Based on HTML5 Algorithm. In International Conference on Frontier Computing (pp. 188-193). Singapore: Springer Nature Singapore.

[9] Kumari, A. S. H. I. S. H., Mittal, M. A. N. A. V., Jha, V. I. P. U. L., Sahu, A. B. H. I. S. H. E. K., Kumar, M. A. N. I. S. H., Sangwan, N. E. E. T. I., & Bohra, N. A. V. D. E. E. P. (2022). Algorithm visualisation— modern web-based visualization of sorting and searching Appl Math Sci, 21.. algorithms. Adv Appl Math Sci, 21.

[10] Lanjewar, R., Mehra, N., Doshi, A., & Mane, Y. (2022). Algovis–An Enhanced Way to Visualize Algorithms. In ITM Web of Conferences (Vol. 44, p. 03013). EDP Sciences.

[11] Kocakoyun, Ş. (2017). Developing of android mobile application using java and eclipse: an application. International Journal of Electronics Mechanical and Mechatronics Engineering, 7(1), 1335-1354.

[12] Karavirta, V., & Shaffer, C. A. (2015). Creating engaging online learning material with the jsav javascript algorithm visualization library. IEEE Transactions on Learning Technologies, 9(2), 171- 183.

[13] Tuparov, G., Tuparova, D., & Jordanov, V. (2014). Teaching sorting and searching algorithms through simulation-based learning objects in an introductory programming course. Procedia-Social and Behavioral Sciences, 116, 2962-2966.

[14] Naps, T. L. (2005). Jhavé: Supporting algorithm visualization. IEEE Computer Graphics and Applications, 25(5), 49-55.

[15] Pierson, W. C., & Rodger, S. H. (1998). Web-based animation of data structures using JAWAA. ACM SIGCSE Bulletin, 30(1), 267-271.

Appendix

Gradle Configuration and Dependencies

1. Project-Level Gradle File (build.gradle)

```
buildscript {
   repositories {
      google()
      mavenCentral()
   }
   dependencies {
      classpath 'com.android.tools.build:gradle:8.11.2'
      classpath 'com.google.gms:google-services:4.4.2'
   }
}
task clean(type: Delete) {
   delete rootProject.buildDir
}
```

2. App-Level Gradle File (build.gradle)

```
plugins {
   id 'com.android.application'
   id 'com.google.gms.google-services'
}
android {
   namespace "com.iiitd.dsavisualizer"
   compileSdkVersion 34
   defaultConfig {
      applicationId "com.iiitd.dsavisualizer"
      minSdkVersion 21
      targetSdkVersion 34
      versionCode 1
      versionName "1.0"
   }
```

```
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-
rules.pro'
        }
    }
}
dependencies {
    implementation fileTree(dir: "libs", include: ["*.jar"])
    implementation 'androidx.appcompat:appcompat:1.3.1'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
    implementation 'com.google.android.material:material:1.12.0'
    implementation 'com.github.florent37:ViewAnimator:v1.1.2'
    implementation 'com.otaliastudios:zoomlayout:1.9.0'
    implementation 'pl.droidsonroids.gif:android-gif-drawable:1.2.22'
    implementation 'de.hdodenhof:circleimageview:3.1.0'
    implementation(platform("com.google.firebase:firebase-bom:32.8.0"))
    implementation("com.google.firebase:firebase-common")
    implementation("com.google.ai.client.generativeai:generativeai:0.9.0")
    implementation 'com.google.guava:guava:31.1-android'
}
```

3. Additional Dependencies for DSA Visualization

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
        maven { url "https://jitpack.io" }
    }
}
dependencyResolutionManagement {
```

```
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven { url "https://jitpack.io" }
    }
}
rootProject.name = "DSA Visualizer"
include(":app")
```

Publication

Gmail

Disha Waghmare <dishawaghmare209@gmail.com>

2026 IEEE International Conference on Emerging Trends in Information, Communication & Systems : Submission (1013) has been created.

1 message

Microsoft CMT <noreply@msr-cmt.org> 26 January 2026 at 17:02
To: dishawaghmare209@gmail.com

Hello,

The following submission has been created.

Track Name: ICETICS2026

Paper ID: 1013

Paper Title: An Offline Mobile-Based App for Visual Learning of Algorithms

Abstract:
DSA and other difficult subjects are now easier to understand thanks to mobile-based learning. When they read books or attend lectures, many students barely understand how algorithms operate. This Android app uses animations to display the steps of any algorithm. By providing clear graphics, pseudocode, and controls to pause, replay, or alter the inputs, it aids users in understanding various sorting, tree, and graph algorithms. Because it is entirely offline and modular in design, students with limited resources will find it very helpful. Students learn DSA more easily and effectively because theoretical concepts are transformed into interactive, visual learning.

Created on: Mon, 26 Jan 2026 11:32:32 GMT

Last Modified: Mon, 26 Jan 2026 11:32:32 GMT

Authors:
- bjkhemani@apsit.edu.in (Primary)
- jdgupta@apsit.edu.in
- ikrasayyad4@gmail.com
- shikshitayadav9502@gmail.com
- dishawaghmare209@gmail.com
- pratikshapathak491@gmail.com

Secondary Subject Areas: Not Entered

Submission Files:
AP_Research_paper_Final.pdf (357 Kb, Mon, 26 Jan 2026 11:29:54 GMT)

Submission Questions Response:
1. Author Agreement for Submission
Agreement accepted

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our Privacy Statement.

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

An Offline Mobile-Based App for Visual Learning of Algorithms

Jaya Gupta
Department of computer science and engineering (AIML)
A.P. Shah Institute of Technology
Thane(W)-400615, India
jdgupta@apsit.edu.in

Bharti Khemani
Department of computer science and engineering (AIML)
A.P. Shah Institute of Technology
Thane(W)-400615, India
bjkhemani@apsit.edu.in

Ikra Sayyad
Department of computer science and engineering (AIML)
A.P. Shah Institute of Technology
Thane(W)-400615, India
ikrasayyad4@gmail.com

Shikshita Yadav
Department of computer science and engineering (AIML)
A.P. Shah Institute of Technology
Thane(W)-400615, India
shikshitayadav9502@gamil.com

Disha Waghmare
Department of computer science and engineering (AIML)
A.P. Shah Institute of Technology
Thane(W)-400615, India
dishawaghmare209@gmail.com

Pratiksha Pathak
Department of computer science and engineering (AIML)
A.P. Shah Institute of Technology
Thane(W)-400615, India
pratikshapathak491@gmail.com

***Abstract*— DSA and other difficult subjects are now easier to understand thanks to mobile-based learning. When they read books or attend lectures, many students barely understand how algorithms operate. This Android app uses animations to display the steps of any algorithm. By providing clear graphics, pseudocode, and controls to pause, replay, or alter the inputs, it aids users in understanding various sorting, tree, and graph algorithms. Because it is entirely offline and modular in design, students with limited resources will find it very helpful. Students learn DSA more easily and effectively because theoretical concepts are transformed into interactive, visual learning.**

Keywords*— *data structures and algorithms, algorithm visualization, android app, mobile-based learning, offline education.

I. Introduction

The foundation of computer science education is Data Structures and Algorithms (DSA). However, these ideas' abstract nature frequently makes comprehension difficult. While algorithms function through dynamic sequences of operations—such as swapping elements, traversing nodes, or updating weights—that are challenging to mentally visualise, textbooks describe how data is arranged.

In order to overcome this, we created this app using Android Studio, the official Integrated Development Environment (IDE) for Android. This instructional mobile application uses animated visualisations to make algorithmic concepts easier to understand. Through real-time execution flow observation, the app enables students to investigate sorting strategies, tree traversals, and graph algorithms.

A. *Need for the Application*

The majority of visualisation tools currently in use are web-based, necessitating desktop browsers and active internet connections, which restricts accessibility for students who are on the go. It fills the need for an interactive, offline, and portable learning tool that students can use at any time to get

ready for competitive programming, technical interviews, or academics.

B. What is DSA and Algorithm Visualization?

Algorithms are the sequential processes used to manipulate data, whereas data structures (such as arrays, trees, and graphs) are ways to store data. The graphical depiction of these processes is called algorithm visualisation. Visualisation enables the learner to observe the "flow" of logic—how comparisons are made, pointers move, and the data structure changes over time—instead of reading static code. Conceptual clarity and logical retention are greatly improved by this visual feedback loop.

C. Overview of Android Studio

Android Studio is a one-stop solution for building Android applications. It provides a complete code editor, layout designer, and testing simulators. For this project, we utilized Android Studio to develop a modular Java-based application that is lightweight and supports multiple devices.

II. Literature survey

We examined current solutions to find gaps in the market before creating this app.

i. Web-Based Solutions: Kumari et al. (2022) and Singh & Patel (2023) proposed web-based platforms for sorting and searching algorithms. While effective, these systems are dependent on continuous internet access and are not optimized for mobile interfaces[1][8]. Pierson and Rodger (1998) developed JAWAA, a web-based system for animating data structures[13]. Naps (2005) developed Jhavé, which assists in the visualization of algorithms in computer science education[14].

ii. Gamification & AI: Bhandari et al. (2025) introduced an AI chatbot integration for DSA learning. While engaging, it lacks a dedicated offline mobile framework[5].

iii. Desktop Applications: Legacy software exists for PCs, but as noted by Zhang (2024), there is a distinct lack of portable tools that allow for revision during commutes or in areas with poor connectivity[4].

iv. Previous Mobile Attempts: Kocakoyun (2019) discussed Android development for education, but few dedicated DSA apps exist that combine Sorting, Trees, and Graphs into a single offline package without intrusive advertisements[10].

Improved Algorithm Visualization Systems: Lanjewar et al. 2022 and Kogan et al. 2024 showed that animated and interactive visualizations improve understanding of data

TABLE 1 Research Gap

Ref & Year	Algorithm	Research Gap
[1] 2022	General Algorithm Animation	Limited to internet access alone; not mobile or offline compatible.
[2] 2022	Sorting, Searching	Focuses on improvement without a modular approach in mobile design.
[3] 2024	Sorting, Recursion, Graphs	Limited coverage of algorithms; no offline support.
[4] 2024	Data Structures and Algorithms	Instruction geared more toward desktops; lacks mobile or offline access.
[5] 2025	Sorting, Searching	Web-based, no offline capability, and no modular expansion.
[6] 2024	General Algorithm Animation	Uses only web technologies; it does not have a mobile or modular extension.
[7] 2024	Intelligent Algorithms and DS	Requires online connectivity; not focused on mobile or offline use.
[8] 2023	Advanced Algorithms	Web-only; no mobile/offline features.
[9] 2023	Sorting, Trees, Graphs	Survey only; no practical implementation
[10] 2019	Android activity lifecycle & event-driven programming	Lack of practical, step-by-step, and scholarly guidelines for beginners.

Identified Gap**:** As per described in Table 1, there is a need for a unified, offline mobile application that

visualizes a wide breadth of algorithms (Sorting, Trees, Graphs) with synchronized pseudocode.

III. Proposed system

This is an Android-based app designed to visualize the internal execution of algorithms.

A. *Features*

- Algorithm Selection Panel: Categorized menu for Sorting, Trees, and Graphs.
- Input Customization: Supports both user-defined
- input and random data generation.
- Step-by-Step Visualization: Animations for swaps, traversals, and pointer movements.
- Pseudocode Highlighting: Highlights the specific line of code executing in sync with the animation.
- Interactive Controls: Play, Pause, Reset, and Speed Control (Slow/Fast).
- Offline Capability: Fully functional without internet connectivity.

B. System *Workflow*

The workflow follows a linear user journey:

- Initialization: User selects a category (for example, Graphs).
- Selection: User selects an algorithm (for example, BFS).
- Input: User provides data or selects "Random".
- Processing: The logic layer calculates steps.
- Visualization: The UI layer renders the steps with pseudocode.

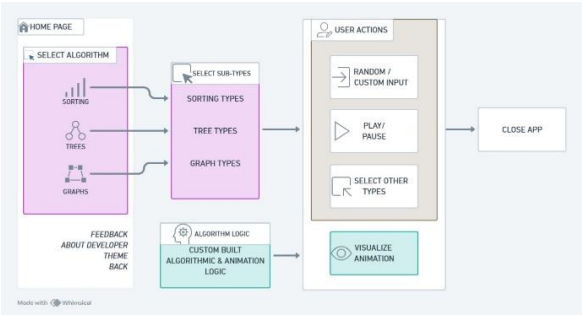


Fig. 1. Workflow

IV. Implementation Methodology

i. Animation Engine: Custom View components were created to render nodes and bars. Transitions were handled using Android's ObjectAnimator and Handler classes to ensure smooth frame updates.

ii. Logic Modules: Each algorithm (for example, QuickSort.java) is encapsulated in a separate class to maintain modularity.

iii. Synchronization: A state-management system links the logical step (array index change) to the UI thread, ensuring the visual update happens exactly when the logic executes.

V. Result

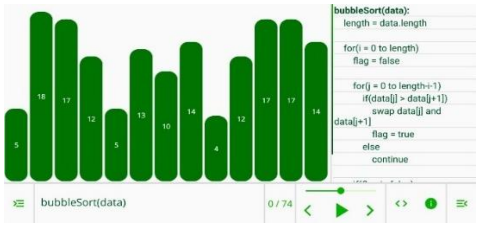


Fig. 2. Before Sorting

Fig 2 is an example of BubbleSort, which shows the initial state before execution. The vertical bars of different heights represent unsorted elements in an array.

Fig. 3. After Sorting

Fig 3 shows the final state after completion of the sorting algorithm execution. Hence, the vertical bars are now sorted in ascending order.

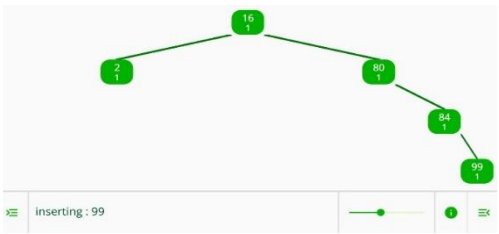


Fig. 4. Unbalanced AVL Tree

Fig 4 indicates the unbalanced AVL tree because the difference between the left sub-tree and the right sub-tree is not -1,0 or 1.

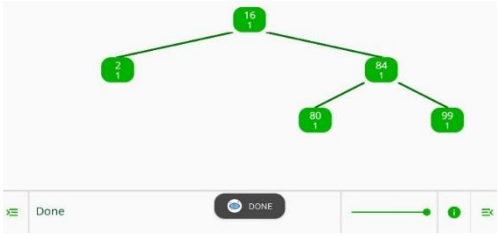


Fig. 5. Balanced AVL Tree

Fig 5 illustrates the balanced AVL tree after performing LL (left-left) rotation which rebalances the tree according to the property of AVL tree.

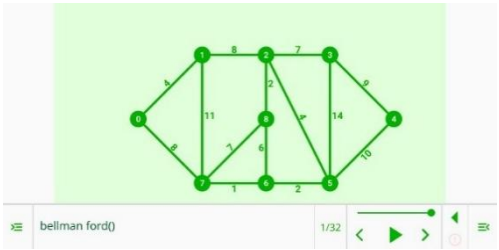


Fig. 6. Graph Visualization

Fig 6 indicates the weighted graph for BellmanFord algorithm, before finding the shortest path where all edges and vertices are in their initial state.

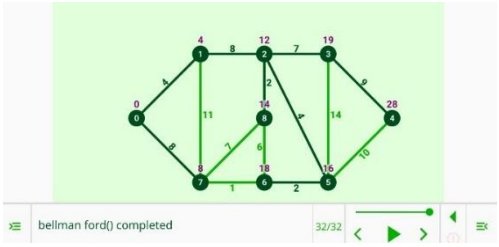


Fig. 7. Shortest Path

Fig 7 shows the highlighted path that is the shortest path from the source vertex to every other vertex in the graph which indicates the minimum cost path obtained after edge relaxation.

VI. Conclusion

DSA is the basis for effective software engineering and problem-solving in computer science. However, students often find it hard to understand these ideas through traditional text-based materials and classroom lectures because algorithmic processes are always changing. This project addresses the gap between theoretical knowledge and practical understanding by providing a portable, offline, and interactive visualization platform. By taking advantage of Android Studio, the application presents step-by-step visual animations that demonstrate how complex algorithms operate internally. Users can observe data transformations in real time, control execution flow through pause and replay options, and experiment with different input values to strengthen conceptual clarity. This project highlights the transformative role of mobile technology in modern technical education by converting abstract DSA concepts into intuitive visual experiences.

REFERENCES

[1] Kumari, A. S. H. I. S. H., Mittal, M. A. N. A. V., Jha, V. I. P. U. L., Sahu, A. B. H. I. S. H. E. K., Kumar, M. A. N. I. S. H., Sangwan, N. E. E. T. I., & Bohra, N. A. V. D. E. E. P. (2022). Algorithm visualisation—modern web-based visualization of sorting and searching Appl Math Sci, 21.. algorithms. Adv Appl Math Sci, 21.

[2] Lanjewar, R., Mehra, N., Doshi, A., & Mane, Y. (2022). Algovis–An Enhanced Way to Visualize Algorithms. In ITM Web of Conferences (Vol. 44, p. 03013). EDP Sciences.

[3] Kogan, G., Chassidim, H., & Rabaev, I. (2024). The efficacy of animation and visualization in teaching data structures: a case study. Educational technology research and development, 72(4), 2349 2372.

[4] Zhang, Yan. (2024). Application of Algorithm Visualization Techniques in Teaching Computer Data Structure Course. Applied Mathematics and Nonlinear Sciences. 9. 10.2478/amns-2024-0788..

[5] Bhandari, S. R., Abhijeet, C., et al., 2025. Algorithm visualization on data structures along with AI chatbot. IJIRCST, 13(1), 21–27.

[6] Liu, G. (2023, July). Research and Application of Animation Visualization Based on HTML5 Algorithm. In International Conference on Frontier Computing (pp. 188-193). Singapore: Springer Nature Singapore.

[7] Lou, G., Sufang, W., & Shen, H. (2023, August). Research on Graphic Interactive Visual Online Teaching of Intelligent Algorithm Course. In EAI International Conference, BigIoT-EDU (pp. 506 516). Cham: Springer Nature Switzerland.

[8] Matalia, Jeet & Kathiriand, Shreya & Patel, Priyanka. (2024). Demystifying Complex Algorithms: A Visual Guide Through the DSA Visualizer. 10.1007/978-981-97-1313-4_18.

[9] Gupta, V., & Rao, D., 2023. Visualizing algorithms in the field of education: A comprehensive review. IJRASET, 11(7), 250–258.

[10] Kocakoyun, Ş. (2017). Developing of android mobile application using java and eclipse: an application. International Journal of Electronics Mechanical and Mechatronics Engineering, 7(1), 1335-1354.

[11] Kumari, A., Bohra, N., & Sangwan, P. Evaluating Software Algorithm Visualization Projects.

[12] Tuparov, G., Tuparova, D., & Jordanov, V. (2014). Teaching sorting and searching algorithms through simulation-based learning objects in an introductory programming course. Procedia-Social and Behavioral Sciences, 116, 2962-2966.

[13] Pierson, W. C., & Rodger, S. H. (1998). Web-based animation of data structures using JAWAA. ACM SIGCSE Bulletin, 30(1), 267-271.

[14] Naps, T. L. (2005). Jhavé: Supporting algorithm visualization. IEEE Computer Graphics and Applications, 25(5), 49-55.

[15] Karavirta, V., & Shaffer, C. A. (2015). Creating engaging online learning material with the jsav javascript algorithm visualization library. *IEEE Transactions on Learning Technologies*, *9*(2), 171-183.

Acknowledgements

We have great pleasure in presenting the report on ***Algopilot (Algorithm Visualizer and Platform Execution).*** We express our sincere gratitude to our guide, **Dr. Jaya Gupta**, for her valuable technical guidance, constant encouragement, and support throughout the development of this project. We also extend our heartfelt thanks to **Prof. Bharthi Khemani** for her guidance in the publication of the research paper and for her insightful suggestions.

We would like to express our deep gratitude to **Dr. Uttam Kolekar**, Principal, for providing us with the opportunity and a supportive environment to carry out this project successfully. We are also sincerely thankful to **Dr. Jaya Gupta** for her encouragement during progress meetings and for her valuable guidance in preparing this report.

We would like to thank our project coordinator, **Prof. Sayali Badhan**, for her constant support, motivation, and guidance throughout the project. We also extend our gratitude to all BE project coordinators, the entire staff, and our colleagues at APSIT for their invaluable help, cooperation, and encouragement.

Shikshita Yadav (22106092)

Disha Waghmare (22106033)

Pratiksha Pathak (22106127)